CentraleSupélec

Mathematics and Data Science

**Research Project Report**

Academic Year 2025-2026

# Probabilistic Detection of Market Manipulation

Adonis JAMAL  Jean-Vincent MARTINI

Supervisor: Damien CHALLET

Academic Tutor: Lionel GABET

April 2026

# Abstract

https://github.com/adonis107/MDS-PDMM

# Acknowledgements

# Contents


**1 Introduction** **6**
1.1 Context and Motivation . . . 6
1.2 Research Questions . . . 7
1.3 Contributions . . . 7
1.4 Report Outline . . . 8

**2 Background and Related Work** **9**
2.1 Limit Order Books and Market Microstructure . . . 9
2.2 Market Manipulation Strategies . . . 9
2.3 Anomaly Detection in Financial Data . . . 10
2.3.1 Reconstruction-Based Approaches . . . 10
2.3.2 Probabilistic Scoring via Synthetic Manipulation . . . 11
2.3.3 One-Class and Density-Based Approaches . . . 11
2.4 Thresholding Methods for Anomaly Detection . . . 12
2.5 Feature Attribution Methods . . . 13

**3 Methodology** **14**
3.1 Feature Engineering . . . 14
3.1.1 Order Book Imbalance . . . 15
3.1.2 Price Dynamics and Microstructure . . . 16
3.1.3 Liquidity and Elasticity . . . 16
3.1.4 Volatility . . . 17
3.1.5 Event Flow and Rapidity . . . 17
3.1.6 Hawkes-Inspired Memory Features . . . 18
3.1.7 Order Flow Imbalance . . . 19
3.1.8 Preprocessing and Scaling . . . 20
3.2 Detection Models . . . 21
3.2.1 Transformer Bottleneck Autoencoder with One-Class SVM . . . 21
3.2.2 Probabilistic Neural Network . . . 24
3.2.3 Probabilistic Robust Autoencoder . . . 26
3.3 Anomaly Thresholding . . . 28
3.3.1 Peak Over Threshold . . . 29
3.3.2 SPOT and DSPOT . . . 30
3.3.3 Rolling False Discovery Rate . . . 32
3.4 Spoofing Gain Estimation . . . 33
3.5 Interpretability . . . 35
3.5.1 Integrated Gradients . . . 36
3.5.2 Grouped Occlusion . . . 37
3.6 Post-Hoc Analysis . . . 38
3.6.1 Anomaly Clustering . . . 38

3.6.2 Jump Analysis . . . 39

**4 Data and Experimental Setup** **41**
4.1 Dataset Description . . . 41
4.1.1 Data Source and Instrument . . . 41
4.1.2 Data Format and Raw Structure . . . 41
4.1.3 Dataset Statistics . . . 41
4.2 Preprocessing Pipeline . . . 42
4.3 Training Protocol . . . 43
4.3.1 Model Hyperparameters . . . 44
4.4 Test Protocol . . . 45
4.5 Compute Environment . . . 46

**5 Results** **47**
5.1 Model Diagnostics . . . 47
5.1.1 Transformer + OC-SVM . . . 48
5.1.2 PNN . . . 48
5.1.3 PRAE . . . 48
5.2 Anomaly Detection on Test Data . . . 48
5.2.1 Anomaly Rates by Model and Time Period . . . 48
5.2.2 Score Time Series . . . 48
5.2.3 Model Agreement . . . 48
5.3 Threshold Comparison . . . 48
5.4 Proximity Analysis . . . 48
5.5 Spoofing Gain Analysis . . . 48
5.6 Sensitivity and Feature Attribution . . . 48
5.6.1 Integrated Gradients: PNN and PRAE . . . 48
5.6.2 Grouped Occlusion: Transformer + OC-SVM . . . 48
5.6.3 Cross-Model Feature Importance . . . 48
5.7 Anomaly Clustering . . . 48
5.8 Jump Analysis . . . 48
5.9 Cross-Year Robustness . . . 48

**6 Discussion** **49**
6.1 Limitations . . . 49
6.2 Generalization and Regime Sensitivity . . . 49
6.3 Comparison of Detection Approaches . . . 49
6.4 Interpretation of Detected Anomalies . . . 49

**7 Conclusion** **50**
7.1 Summary of Contributions . . . 50
7.2 Future Work . . . 50

**A Feature Catalogue** **52**

**B Hyperparameter Tables** **53**

**C Additional Figures** **54**
C.1 Score Distributions per Test Day . . . 54
C.2 LOB Snapshots for Selected Anomalies . . . 54
C.3 Feature Evolution Around Anomalies . . . 54

**D Threshold Method Details** **55**

**E Spoofing Gain Derivation** **56**

# Chapter 1

# Introduction

## 1.1 Context and Motivation

Modern securities exchanges organise trading through electronic limit order books. A limit order book (LOB) records all outstanding buy and sell orders at each price level, providing a real-time, public summary of supply and demand. The instantaneous state of the book, defined by the distribution of volume across price levels on both sides, and the recent flow of incoming orders jointly determine short-term price dynamics. Tao et al. [13] model this dependence through a multi-level volume imbalance that aggregates supply and demand across several price levels, showing on TMX data that imbalance shifts are directly linked to the probability distribution of subsequent price moves. Fabre and Challet [5] extend this line of work by introducing Hawkes-inspired order flow features that capture both the size and the distance of placement of new limit orders, demonstrating on cryptocurrency data that these features predict the conditional distribution of mid-price movements.

Market manipulation exploits this dependence. By artificially altering either the instantaneous book state or the order flow, a manipulator can induce a predictable price response and profit from it. Spoofing, the primary focus of this project, consists of placing large visible limit orders with no intention of execution. These orders shift the perceived supply-demand balance, move the mid-price in the desired direction, and are cancelled once the manipulator's genuine order on the opposite side has been filled. Layering extends spoofing by stacking multiple non-bona-fide orders across several price levels, amplifying the perceived depth on one side of the book [5]. Both strategies distort the volume imbalance: Tao et al. [13] show that the optimal spoofing action shifts the multi-level imbalance away from its equilibrium value, and that comparing the observed imbalance before a market order to its long-run distribution provides a basis for detection.

The regulatory treatment of these strategies differs sharply between asset classes. In regulated equity and derivatives markets, spoofing and layering are explicitly prohibited under the European Market Abuse Regulation (EU MAR) and the US Dodd-Frank Act. In cryptocurrency markets, the absence of equivalent regulation opens the door to manipulative behaviour on a larger scale [5]. Fabre and Challet, working on Level-3 crypto order book data, find that 31% of large orders on the BTC-USD and ETH-USD pairs could profitably spoof the market over a four-day period. This asymmetry suggests that crypto order book data should contain a substantially higher base rate of manipulative patterns than equity data. The present work focuses on the more challenging equity setting, where manipulation is rarer and must be detected without labels.

**Why unsupervised, score-based detection.** A binary classifier that labels each event as "manipulative" or "not" is poorly suited to this detection problem for three reasons. First, no ground-truth labels exist: in equity markets, confirmed manipulation cases are rare and not publicly annotated at the order level. Second, manipulation is not a discrete category but a spectrum of intent, from aggressive but legitimate liquidity provision to unambiguous spoofing. Third, a probabilistic score is more actionable than a binary flag: it allows a surveillance system to rank events by severity and calibrate alert thresholds to a desired false discovery rate. Fabre and Challet [5] address these requirements with a probabilistic neural network that outputs a full predictive distribution over future price moves, from which both an anomaly score (the negative log-likelihood) and an economic spoofing gain estimate (formalised in Section 3.4) can be derived.

## 1.2 Research Questions

This project investigates the following questions:

1. Can unsupervised probabilistic models trained on order book features detect anomalous order flow in the absence of ground-truth labels?
2. Which order book features carry the most information about anomalous events, and do different detection models agree on the same discriminative features?
3. How should anomaly thresholds be calibrated in a non-stationary, streaming setting when no labelled data are available?
4. Can the economic profitability of a spoofing strategy be estimated directly from model predictions, providing an interpretable measure of manipulation opportunity?
5. Do detection patterns remain stable across different time periods, when models trained on one year are applied to data from other years?

These questions are addressed on the TOTF.PA (TotalEnergies, Euronext Paris) limit order book dataset, covering trading days from 2009, 2010, 2015, and 2017.

## 1.3 Contributions

The main contributions of this report are:

1. A modular detection pipeline that combines three complementary unsupervised models on the same equity LOB dataset (TOTF.PA, Euronext Paris): a Transformer bottleneck autoencoder inspired by Poutré et al. [10], coupled with a Nyström-approximated One-Class SVM in latent space; a Probabilistic Neural Network (PNN) that predicts skew-normal price move distributions following the framework of Fabre and Challet [5]; and a Probabilistic Robust Autoencoder (PRAE) with learnable per-sample gates [7]. Each model produces a scalar anomaly score per time step; their agreement identifies the highest-confidence anomalies.
2. A feature engineering library of approximately 80 features synthesising contributions from multiple sources: Hawkes-inspired memory features with spatial and temporal decay [5], weighted multi-level volume imbalance [13], event flow and cancellation rapidity [10], multi-level order flow imbalance, and standard microstructural indicators (spread, volatility, elasticity). This represents an input space roughly six times larger than that of Poutré et al. [10], who use approximately 13 Level-1 features.

3. Application of the spoofing gain framework of Fabre and Challet [5] to equity LOB data. The PNN-predicted skew-normal parameters feed an expected cost model that quantifies the profitability of a hypothetical spoofing strategy at each time step, producing an interpretable economic measure of manipulation opportunity.
4. A systematic comparison of four threshold methods for converting continuous anomaly scores into binary alerts without labels: Peak Over Threshold (POT), Streaming POT (SPOT), Drift-aware Streaming POT (DSPOT) [11], and a rolling Benjamini-Hochberg False Discovery Rate controller (RFDR).
5. Interpretability analysis via Integrated Gradients [12] for the PNN and PRAE, and Grouped Occlusion for the Transformer+OC-SVM, identifying which feature families and which sides of the order book drive each model's anomaly scores.
6. Post-hoc characterisation of detected anomalies through HDBSCAN clustering of the three anomaly scores and microstructural jump analysis (bipower variation, variance ratio, CUSUM), providing an unsupervised taxonomy of anomaly types.
7. Cross-year robustness evaluation: models trained on 2015 data are tested on 2009, 2010, and 2017 data, and models trained on 2017 data are tested on the same periods, assessing whether the pipeline detects consistent patterns across different market regimes.

## 1.4 Report Outline

Chapter 2 reviews the literature on limit order book microstructure, market manipulation, and the detection methods that this project builds on. Chapter 3 formalises the feature engineering pipeline, the three detection models with their loss functions, the threshold methods, the spoofing gain framework, and the interpretability and post-hoc analysis tools. Chapter 4 describes the TOTF.PA dataset, the preprocessing steps, the sequential training protocol, and the experimental setup. Chapter 5 presents the experimental findings: model diagnostics, anomaly detection rates, threshold comparisons, sensitivity analysis, anomaly clustering, jump analysis, and cross-year robustness. Chapter 6 discusses the limitations of the approach, compares the three detection models, and interprets the detected anomalies in light of known manipulation signatures. Chapter 7 summarises the contributions and outlines directions for future work.

# Chapter 2

# Background and Related Work

This chapter provides the conceptual vocabulary and critical perspective needed to evaluate the methodological choices presented in Chapter 3. Each section introduces a class of approaches, explains what it is good at, where it struggles, and what assumptions it makes. Formal definitions, model architectures, and training procedures are deferred to Chapter 3.

## 2.1 Limit Order Books and Market Microstructure

A limit order book records all outstanding buy and sell orders for a security, organised by price and submission time. At time $t$, the book state consists of $L$ price levels on each side. The bid (buy) side lists prices $p_1^b > p_2^b > \cdots > p_L^b$ with corresponding volumes $v_1^b, v_2^b, \ldots, v_L^b$. The ask (sell) side lists prices $p_1^a < p_2^a < \cdots < p_L^a$ with volumes $v_1^a, v_2^a, \ldots, v_L^a$. The best bid is $p_1^b$, the best ask is $p_1^a$, the mid-price is $p_{\text{mid}} = (p_1^a + p_1^b)/2$, and the spread is $s = p_1^a - p_1^b$.

Three event types modify the book state. A *limit order* adds volume at a specified price level, increasing liquidity. A *cancellation* removes a previously placed limit order, reducing liquidity at that level. A *market order* (equivalently, a trade) consumes volume at the best price on the opposite side, removing liquidity and potentially shifting the best price.

The instantaneous distribution of volume across price levels and the temporal sequence of these events jointly determine short-term price dynamics. Volume imbalance, the normalised excess of bid over ask volume at a given level, is a widely used summary statistic of the book state. Tao et al. [13] extend the single-level imbalance to a weighted multi-level measure that aggregates supply and demand across several price levels, and show on TMX (Toronto Stock Exchange) data that this quantity is directly linked to the probability distribution of subsequent price moves. Fabre and Challet [5] complement imbalance with Hawkes-inspired features that capture both the size and the spatial placement depth of new limit orders, demonstrating on cryptocurrency data that these features predict the conditional distribution of mid-price movements.

## 2.2 Market Manipulation Strategies

Spoofing consists of submitting large, visible limit orders with no intention of execution to create a false impression of supply or demand. The lifecycle of a spoofing event follows a characteristic sequence: (1) the manipulator places one or more large limit orders on one side of the book, shifting the perceived imbalance; (2) the mid-price moves in the direction favoured by the manipulator; (3) the manipulator executes a genuine order on the opposite side at the improved price; (4) the non-bona-fide orders are cancelled. Layering extends spoofing by

stacking multiple orders across several price levels on the same side, amplifying the perceived depth [5]. Quote stuffing is a related strategy in which a large number of orders are rapidly submitted and cancelled to flood the matching engine with messages, degrading other participants' ability to process market data and exploiting the resulting informational advantage. Unlike spoofing, quote stuffing operates primarily through message-rate pressure rather than through direct manipulation of the perceived volume imbalance.

Both spoofing and layering are prohibited under the European Market Abuse Regulation (EU MAR, Article 12) and the US Dodd-Frank Act (Section 747). Detection is challenging because the same order flow patterns can arise from legitimate trading strategies, such as aggressive liquidity provision or inventory management. Tao et al. [13] formalise the detection problem by modelling the imbalance shift caused by a spoof order and comparing the observed pre-trade imbalance distribution to its long-run counterpart via the Wasserstein distance. Fabre and Challet [5] take an economic approach: rather than asking whether an order flow pattern looks abnormal, they ask whether a rational spoofer could profit from the current book state, defining the spoofing gain as the difference in expected execution cost between a scenario where the spoof order is present and one where it is absent.

The arguments for unsupervised, score-based detection over binary classification are detailed in Section 1.1 and are not repeated here.

## 2.3 Anomaly Detection in Financial Data

In the absence of ground-truth labels for manipulation, detection must proceed unsupervised. A central difficulty shared by all unsupervised approaches is the *contamination problem*: training data collected from a real market inevitably contains some anomalous events, and their presence can degrade the learned model of normality. This section surveys three families of approaches and discusses their relative strengths and weaknesses for the LOB anomaly detection problem.

### 2.3.1 Reconstruction-Based Approaches

Autoencoders learn a compressed representation of the training data and reconstruct each input from that representation. When the training set is predominantly normal, the model learns to reconstruct normal patterns well; anomalous inputs produce high reconstruction error because they deviate from the learned structure. This family has several merits: it requires no labels, captures complex multivariate dependencies, and generalises across instruments and time periods without task-specific supervision.

For order book sequences, the choice of encoder architecture matters. Convolutional and recurrent architectures capture local patterns effectively but struggle with long-range dependencies between distant price levels or between events separated by many time steps. The Transformer architecture addresses this limitation through self-attention, which allows each position in the sequence to attend directly to every other position. Multi-head attention enables the model to simultaneously capture different aspects of the book state, such as volume dynamics at the top of the book, spread behaviour, and order arrival patterns deeper in the queue. Poutré et al. [10] demonstrate on NASDAQ LOBSTER data that a Transformer autoencoder combined with a one-class classifier in the latent space outperforms the reconstruction error alone for detecting synthetically injected manipulation patterns (quote stuffing, layering, and pump-and-dump). Notably, their feature set is built from Level-1 (best bid/ask) data only, yielding approximately 13 features; the present work extends this approach to Level-2 (ten price levels per side) data with roughly 80 features, which introduces both richer information and a substantially higher-dimensional input space.

A limitation of deterministic autoencoders is that the reconstruction error is a noisy proxy for anomaly severity: it does not distinguish between inputs that are merely unusual and inputs that fall outside the training distribution entirely. A probabilistic formulation, in which the autoencoder outputs a distribution over reconstructions rather than a point estimate, addresses this limitation by providing an uncertainty signal. Lindenbaum et al. [7] go further: rather than modifying the reconstruction itself, they introduce learnable per-sample gates that allow the training procedure to downweight suspected anomalies, so that the autoencoder focuses on reconstructing clean data even when the training set is contaminated. This makes the model robust to the contamination problem without requiring an explicit data-cleaning step. The degree of contamination in the specific TOTF.PA dataset is unknown; the PRAE's robustness is therefore a precautionary measure rather than a response to a measured contamination rate.

### 2.3.2 Probabilistic Scoring via Synthetic Manipulation

An alternative to asking "does this look abnormal?" is to ask "how much could a rational spoofer gain from this book state?" Fabre and Challet [5] pursue this reframing by training a probabilistic neural network that predicts the full conditional distribution of the next mid-price move. The predicted distribution serves two purposes: its negative log-likelihood provides a general anomaly score (events where the realised price move is unlikely under the model receive high scores), and its parameters feed an expected cost model that quantifies the profitability of a hypothetical spoofing strategy at each time step.

The merits of this approach are substantial. The spoofing gain score has a direct economic interpretation, which makes it more actionable for a market surveillance system than a generic reconstruction error. Training on the predictive task sidesteps the need for labelled manipulation data: the model learns normal price dynamics, and deviations from that model are flagged regardless of their cause. The probabilistic output naturally quantifies uncertainty over the predicted price move, which matters for heavy-tailed financial data where point predictions are unreliable.

The limitations are equally clear. The spoofing gain depends on a model of the spoofer's strategy (order size, placement distance, fee structure), so it may miss manipulation patterns that deviate from that model. The approach was developed and evaluated on cryptocurrency LOB data (BTC-USD and ETH-USD on Coinbase) [5], where the absence of anti-manipulation regulation results in a higher base rate of manipulative behaviour; its effectiveness on regulated equity markets, where manipulation is rarer and subtler, has not been established in the literature.

### 2.3.3 One-Class and Density-Based Approaches

One-class classification methods learn a tight boundary around the training data in a transformed feature space and flag anything outside that boundary as anomalous. The One-Class SVM (OC-SVM) is the most widely used instance: it finds a hyperplane in a kernel-induced feature space that separates the data from the origin with maximum margin, controlled by a parameter $\nu$ that bounds the fraction of training points allowed outside the boundary.

The strengths of the OC-SVM are computational efficiency and theoretical simplicity. It does not require specifying what an anomaly looks like, only what normal data looks like, and the kernel trick allows it to capture non-linear structure in the input space. However, it does not scale well to high-dimensional feature spaces without careful kernel design: in the approximately 80-dimensional feature space typical of LOB anomaly detection, the RBF kernel becomes poorly calibrated and the decision boundary may either collapse or become too loose. Low-rank kernel approximations (such as the Nyström method) partially address this issue by

projecting the kernel evaluation into a tractable subspace, enabling GPU-accelerated training. A further limitation is that the OC-SVM does not produce a calibrated probability; its decision function is a signed distance, not a likelihood, which complicates downstream interpretation and threshold calibration.

For the LOB problem, the OC-SVM is most effective when applied not to raw features but to a learned latent representation. Poutré et al. [10] demonstrate this by fitting an OC-SVM in the bottleneck space of a Transformer autoencoder, where the reduced dimensionality and the learned structure of the representation make the kernel boundary more discriminative. This two-stage approach combines the representation power of reconstruction-based methods with the boundary-learning capacity of one-class classification.

## 2.4 Thresholding Methods for Anomaly Detection

Any continuous anomaly score requires a threshold to produce a detection decision. The naive approach, a fixed percentile of the score distribution, assumes that the distribution is stationary and well-behaved. In financial data, score distributions are heavy-tailed and non-stationary: a fixed percentile will produce too many false positives during volatile periods and miss anomalies during quiet periods. Setting thresholds without labels is therefore a problem in its own right.

**Extreme Value Theory (EVT).** EVT models the tail of the score distribution rather than the bulk, fitting a Generalised Pareto Distribution to the exceedances above a high initial threshold. This is well-suited to heavy-tailed data because the statistical guarantee applies specifically to extreme quantiles, where fixed-percentile methods are least reliable. Siffer et al. [11] extend the batch EVT estimator to streaming settings with two algorithms: SPOT, which updates the tail model as new peaks arrive, and DSPOT, which adds a drift correction that tracks shifts in the baseline score level over time. The main assumption is that the tail of the score distribution is well-approximated by the GPD, which must be verified empirically for each model output.

**False Discovery Rate (FDR) Control.** FDR-based procedures, such as the Benjamini-Hochberg method, control the expected proportion of false positives among all flagged events. This is appropriate when anomalies are rare and the cost of false positives is high, because it bounds the discovery error rate rather than the absolute false positive count. The requirement is a set of p-values or a reference null distribution for the anomaly score, which must be estimated from the data when no parametric form is available.

**Rolling FDR (RFDR).** RFDR adapts the Benjamini-Hochberg procedure to a streaming setting by maintaining a sliding window of recent anomaly scores. Within each window, scores are log-transformed, converted to robust z-scores using the median absolute deviation, and mapped to p-values under a Gaussian assumption; the BH procedure then determines a dynamic threshold that controls the false discovery rate at level $\alpha$. Because the null distribution is re-estimated at every step from the local window, RFDR tracks non-stationarity in the score distribution without requiring an explicit drift model.

**Implicit and Post-Hoc Model Thresholds.** Some detection methods appear to produce a threshold as a byproduct of training. The OC-SVM decision boundary, for instance, defines a binary classification in the feature space: points with negative decision function values are labelled as outliers. However, in practice this binary output is often discarded in favour of a continuous dissimilarity score, and a separate threshold is selected post-hoc on labelled evaluation data. Poutré et al. [10] follow this approach, choosing the threshold that maximises the $F_4$-score on a test set containing synthetic manipulations. This converts an ostensibly unsupervised method into one that depends on labelled data for threshold calibration.

## 2.5 Feature Attribution Methods

A detection score alone is not actionable for a market surveillance analyst: the analyst needs to know which features of the order book drove the score and whether those features align with known manipulation signatures. Attribution for sequential financial data is harder than for static inputs because features at different time steps and price levels are structurally correlated, and because the underlying distribution shifts over the trading day.

Two broad families of attribution methods are relevant. Gradient-based methods, such as Integrated Gradients [12], accumulate the model's gradient along a path from a baseline input to the actual input, producing per-feature importance scores that are theoretically principled (they satisfy completeness and sensitivity axioms) and computationally tractable. Their limitation is that they require a differentiable end-to-end model, which excludes pipelines where a non-differentiable component (such as an OC-SVM) sits between the learned representation and the final score.

For non-differentiable pipelines, ablation-based methods provide an alternative. Grouped occlusion replaces semantically related groups of features (all bid-side features, all Hawkes features, all features at a given price level) with a baseline value and measures the resulting change in the anomaly score. This approach is model-agnostic and handles the structured correlation among LOB features by operating at the group level rather than the individual feature level. The trade-off is lower resolution: it identifies which feature families matter, not which individual features within a family are most important.

# Chapter 3

# Methodology

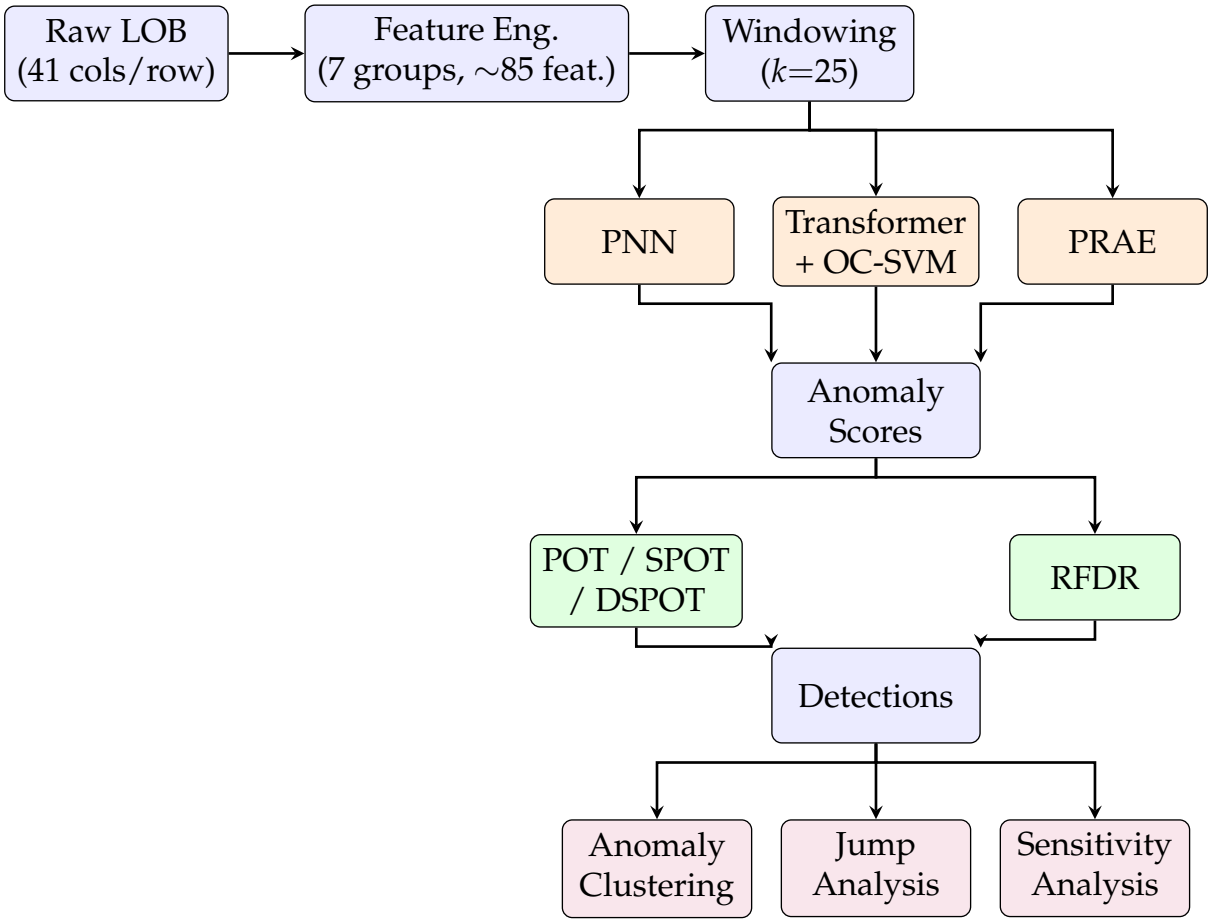


Figure 3.1: End-to-end detection pipeline. Raw LOB snapshots are transformed into ∼85 features, windowed into sequences of length $k$=25, and scored by three complementary models. Threshold methods produce binary detections, which are analysed post hoc via clustering, jump tests, and integrated-gradient attribution.

## 3.1 Feature Engineering

The raw limit order book data for TOTF.PA (TotalEnergies, Euronext Paris) records the state of the book at each update event during continuous trading hours (09:00-17:30 CET). Each row

contains an Excel serial timestamp $t$ (column `xltime`, where the fractional part encodes the time of day) and the price and volume at $L = 10$ levels on each side: $p_1^b, V_1^b, \ldots, p_{10}^b, V_{10}^b$ for the bid and $p_1^a, V_1^a, \ldots, p_{10}^a, V_{10}^a$ for the ask, giving 41 columns per row. The data is event-driven: a new row is recorded whenever a limit order, cancellation, or trade modifies the visible book state, so the inter-event spacing is irregular.

Feeding the 40 raw price-volume columns directly to a model is wasteful and fragile. Prices at adjacent levels are highly correlated, volumes are non-stationary, and the absolute price level carries no information about short-term dynamics. Engineered features reduce the input dimensionality to $d_{\text{in}} \approx 80$ informative quantities that are more stationary, more interpretable, and encode the domain knowledge needed to distinguish normal from anomalous order flow.

The features are organised into seven groups: (1) order book imbalance, capturing supply-demand asymmetry across depth levels; (2) price dynamics and microstructure, summarising price movements and their relationship to volume; (3) liquidity and elasticity, measuring the cost and shape of the book; (4) volatility proxies; (5) event flow and rapidity, characterising the intensity and type of order arrivals; (6) Hawkes-inspired memory features, capturing self-exciting patterns in order flow with spatial and temporal decay; and (7) order flow imbalance, measuring signed directional pressure from volume changes conditioned on price movements. Preprocessing and scaling are described in Section 3.1.8.

### 3.1.1 Order Book Imbalance

Imbalance features quantify the asymmetry between supply and demand at various depths of the book. Spoofing and layering distort this asymmetry by injecting volume on one side; imbalance measures are therefore among the most direct indicators of manipulative intent.

The Level-1 imbalance compares bid and ask volume at the best quotes:

$$I_t^{(1)} = \frac{V_{1,t}^b - V_{1,t}^a}{V_{1,t}^b + V_{1,t}^a}. \tag{3.1}$$

Values near $+1$ indicate strong buy-side pressure; values near $-1$ indicate sell-side pressure.

The Level-5 imbalance aggregates volume across the top five levels on each side:

$$I_t^{(5)} = \frac{\sum_{i=1}^5 V_{i,t}^b - \sum_{i=1}^5 V_{i,t}^a}{\sum_{i=1}^5 V_{i,t}^b + \sum_{i=1}^5 V_{i,t}^a}. \tag{3.2}$$

A divergence between $I_t^{(1)}$ and $I_t^{(5)}$ suggests that volume is concentrated away from the best quotes, a pattern consistent with layering.

To weight deeper levels more flexibly, we follow Tao et al. [13] and define a weighted multi-level imbalance over $L = 5$ levels with weight vector $\mathbf{w} = (w_1, \ldots, w_5)$:

$$I_t^{(\mathbf{w})} = \frac{\sum_{i=1}^5 w_i V_{i,t}^b}{\sum_{i=1}^5 w_i V_{i,t}^b + \sum_{i=1}^5 w_i V_{i,t}^a}. \tag{3.3}$$

This quantity lies in $[0, 1]$; values above 0.5 indicate net bid-side weight. Three weight configurations are used: *deep-weighted* $\mathbf{w} = (0.1, 0.1, 0.2, 0.2, 0.4)$, which emphasises deeper levels where spoof orders are less likely to be executed; *top-weighted* $\mathbf{w} = (0.4, 0.2, 0.2, 0.1, 0.1)$, which emphasises the best quotes; and *uniform* $\mathbf{w} = (0.2, 0.2, 0.2, 0.2, 0.2)$, which weights all five levels equally. Unlike the Level-1 volume imbalance used by Poutré et al. [10], this weighted formulation aggregates information across five price levels, so $I_t^{(\mathbf{w})}$ reflects the depth structure of the book rather than only the best quotes.

### 3.1.2 Price Dynamics and Microstructure

This group captures the immediate state of the price process and its first- and second-order dynamics.

The mid-price and bid-ask spread are:

$$p_{\text{mid},t} = \frac{p^a_{1,t} + p^b_{1,t}}{2}, \qquad s_t = p^a_{1,t} - p^b_{1,t}. \tag{3.4}$$

The micro-price deviation measures how far the L1-imbalance-adjusted fair value lies from the mid-price:

$$\delta^{\text{micro}}_t = I^{(1)}_t \cdot \frac{s_t}{2}. \tag{3.5}$$

A large deviation suggests that the balance of volume at the best quotes is pulling the fair value away from the geometric centre of the spread.

The volume depth ratio measures the concentration of volume away from the best quote on each side $s \in \{b, a\}$:

$$D^s_t = \frac{\sum_{i=2}^{5} V^s_{i,t}}{V^s_{1,t}}. \tag{3.6}$$

High values indicate that most liquidity sits behind the best quote, a signature of layering where large volumes are placed at deeper levels.

Log returns, velocity, and acceleration of the mid-price are:

$$r_t = \ln\left(\frac{p_{\text{mid},t}}{p_{\text{mid},t-1}}\right), \quad v_t = p_{\text{mid},t} - p_{\text{mid},t-1}, \quad a_t = v_t - v_{t-1}. \tag{3.7}$$

The net order flow at Level 1 isolates volume changes that occur at a fixed price, filtering out changes caused by price-level shifts:

$$F^s_t = \Delta V^s_{1,t} \cdot \mathbf{1}\left[p^s_{1,t} = p^s_{1,t-1}\right], \qquad s \in \{b, a\}, \tag{3.8}$$

where $\Delta V^s_{1,t} = V^s_{1,t} - V^s_{1,t-1}$. A large positive $F^b_t$ followed by a rapid cancellation is a textbook spoofing pattern.

Volume deltas $\Delta V^b_{1,t}$ and $\Delta V^a_{1,t}$ at the best quotes are also retained as raw features, along with the rolling standard deviation of the mid-price over a window of $W = 50$ events.

### 3.1.3 Liquidity and Elasticity

These features describe how much it costs to execute a given order size and how steeply the book's price schedule rises with volume.

The sweep-to-fill cost is the volume-weighted average price paid to execute a target order of size $Q = 1{,}000$ units by walking through $L = 10$ levels. The value $Q = 1{,}000$ was chosen empirically to represent a moderately sized institutional order for TOTF.PA; no reference paper prescribes a specific target size, and a sensitivity analysis over $Q$ is deferred to Chapter 5. Denoting by $C^s_t(Q)$ the resulting average execution price on side $s$, the reported features are the excess costs relative to the mid-price:

$$\text{SC}^a_t = C^a_t(Q) - p_{\text{mid},t}, \qquad \text{SC}^b_t = p_{\text{mid},t} - C^b_t(Q). \tag{3.9}$$

Both quantities are non-negative under normal conditions. Large values indicate thin liquidity: a spoofing episode that removes genuine orders from the book can cause a spike in sweep cost.

The book slope (elasticity) measures the sensitivity of price to cumulative volume. For each side $s$, a linear regression of price on log cumulative volume across the first $D = 5$ levels gives the following relationship (Tao et al. [13] calibrate a comparable depth parameter to values between 3 and 5 depending on the stock; Fabre and Challet [5] use $K = 5$ as a representative depth): The depth $D = 5$ is consistent with the range used by Tao et al. [13] (who calibrate 3 to 5 depending on the stock) and Fabre and Challet [5] (who use $K = 5$); an ablation over $D$ is reported in Chapter 5.

$$p^s_{i,t} \approx \alpha^s_t + \beta^s_t \ln\left(\sum_{j=1}^{i} V^s_{j,t} + 1\right), \qquad i = 1, \ldots, D, \tag{3.10}$$

and the feature is the absolute slope $|\beta^s_t|$. A steep slope (large $|\beta^s_t|$) indicates low liquidity, where a small volume would cause a large price change; a flat slope indicates a deep, resilient book.

### 3.1.4 Volatility

Local volatility features capture the recent variability of the price process and of event timing.

Rolling standard deviations of log returns are computed over two horizons:

$$\sigma^{(W)}_t = \sqrt{\frac{1}{W-1}\sum_{k=0}^{W-1}(r_{t-k} - \bar{r}_t)^2}, \qquad W \in \{50, 100\}. \tag{3.11}$$

The normalised price range over a window of $W = 50$ events measures extreme movements:

$$R_t = \frac{\max_{k\in[t-W+1,t]} p_{\text{mid},k} - \min_{k\in[t-W+1,t]} p_{\text{mid},k}}{p_{\text{mid},t}}. \tag{3.12}$$

A sudden spike in $R_t$ without a corresponding increase in $\sigma^{(50)}_t$ may indicate a transient price dislocation caused by manipulation.

The absolute velocity $|r_t|$ captures the instantaneous magnitude of price movement without directional information.

The inter-event time $\Delta t_t = t - t_{t-1}$ (in fractions of a day, following the Excel `xltime` convention) and its logarithm $\ln(\Delta t_t + \epsilon)$, with $\epsilon = 10^{-6}$, are included to provide the models with information about event timing. Abnormally short inter-event times can signal quote-stuffing activity.

### 3.1.5 Event Flow and Rapidity

Following Poutré et al. [10], each LOB update is classified as either a cancellation or a trade based on observable price and volume changes. A cancellation on side $s$ is identified when the best price is unchanged and volume decreases ($p^s_{1,t} = p^s_{1,t-1}$ and $\Delta V^s_{1,t} < 0$). A trade on side $s$ is labelled by the *aggressor* (the marketable order that initiates the transaction): a buy trade is detected when the ask price changes ($p^a_{1,t} \neq p^a_{1,t-1}$), and a sell trade when the bid price changes ($p^b_{1,t} \neq p^b_{1,t-1}$). The corresponding event-size features are indexed by the *passive* side of the book that absorbed the order ($s = b$ for a buy trade, $s = a$ for a sell trade).

The event size features record the magnitude of each event type:

$$\text{Size}^{\text{cancel},s}_t = |\Delta V^s_{1,t}| \cdot \mathbf{1}[\text{cancel}^s_t], \qquad \text{Size}^{\text{trade},s}_t = V^s_{1,t-1} \cdot \mathbf{1}[\text{trade}^s_t]. \tag{3.13}$$

Each of these four quantities, along with the raw Level-1 volume, is smoothed with a simple moving average over $W_{\text{SMA}} = 10$ events, following the feature definitions in Poutré et al. [10]. The window size $W_{\text{SMA}} = 10$ follows the implementation of Poutré et al. [10], who adopt this value without ablation; we retain it unchanged and note that no sensitivity analysis over $W_{\text{SMA}}$ is performed in this work.

Rapidity features measure the density of events per unit of calendar time:

$$\rho_t^{\text{type},s} = \frac{\mathbf{1}[\text{type}_t^s]}{\Delta t_t}, \qquad \text{type} \in \{\text{cancel}, \text{trade}\}, \quad s \in \{b, a\}. \tag{3.14}$$

High cancellation rapidity on one side of the book is a signature of quote stuffing, where a manipulator floods the exchange with rapid submissions and cancellations.

Bid- and ask-side log returns ($\ln(p_{1,t}^s / p_{1,t-1}^s)$ for $s \in \{b, a\}$) and price speed (log return divided by $\Delta t_t$) complete this group.

### 3.1.6 Hawkes-Inspired Memory Features

Following Fabre and Challet [5], we define order flow variables inspired by the self-exciting structure of Hawkes processes. Let $(N_t^{L,b})_{t\geq 0}$ and $(N_t^{L,a})_{t\geq 0}$ denote the point processes counting the arrivals of buy and sell limit orders, and let $(N_t^{M,b})_{t\geq 0}$ and $(N_t^{M,a})_{t\geq 0}$ denote the point processes of buy and sell marketable orders respectively (using the aggressor-side convention: $N^{M,b}$ counts buy market orders that consume ask-side liquidity, and $N^{M,a}$ counts sell market orders that consume bid-side liquidity). Each limit order carries two marks: a volume $v_t^{L,s} > 0$ and a distance of placement $\delta_t^{L,s} > 0$ from the best price, for $s \in \{b, a\}$. Each marketable order carries a volume mark $v_t^{M,s} > 0$.

The limit-order and marketable-order flow variables are defined as

$$L_t^s(\phi) = \int_{(-\infty,t]} \phi(t-u,\, v_u^{L,s},\, \delta_u^{L,s})\, dN_u^{L,s}, \qquad s \in \{b, a\}, \tag{3.15}$$

$$M_t^s(\psi) = \int_{(-\infty,t]} \psi(t-u,\, v_u^{M,s})\, dN_u^{M,s}, \qquad s \in \{b, a\}, \tag{3.16}$$

where $\phi$ is a positive kernel that weights each past limit order by its elapsed time, volume, and distance of placement, and $\psi$ is a positive kernel that weights each past marketable order by its elapsed time and volume.

The kernels take exponential form with a linear volume function $f(v) = v$:

$$\phi_{\beta,\eta}(t, v, \delta) = e^{-\beta t}\, v\, e^{-\eta\delta}, \qquad \psi_\beta(t, v) = e^{-\beta t}\, v, \tag{3.17}$$

where $\beta > 0$ controls the temporal decay rate and $\eta \geq 0$ controls the spatial decay rate. The exponential time kernel is chosen for two reasons: it satisfies the Markov property, which permits efficient recursive updates in a streaming setting, and combining multiple time scales $\beta$ approximates the slowly decaying empirical memory of order flow across several decades [5].

Two families of kernels are constructed by varying $\beta$ over a finite set $\boldsymbol{\beta}$ and $\eta$ over a finite set $\boldsymbol{\eta}$:

$$\boldsymbol{\phi} = \{\phi_{\beta,\eta} : \beta \in \boldsymbol{\beta},\, \eta \in \boldsymbol{\eta}\}, \qquad \boldsymbol{\psi} = \{\psi_\beta : \beta \in \boldsymbol{\beta}\}. \tag{3.18}$$

The corresponding feature sets at time $t$ are

$$\mathcal{L}_t = \big\{(L_t^b(\phi),\, L_t^a(\phi)) : \phi \in \boldsymbol{\phi}\big\}, \qquad \mathcal{M}_t = \big\{(M_t^b(\psi),\, M_t^a(\psi)) : \psi \in \boldsymbol{\psi}\big\}. \tag{3.19}$$

Each element of $\mathcal{L}_t$ captures limit-order flow at a specific combination of time and distance scales; each element of $\mathcal{M}_t$ captures marketable-order flow at a specific time scale. Small $\eta$ values give nearly equal weight to orders at all depths; large $\eta$ values concentrate attention on orders near the best quotes, so that distant spoof orders receive negligible weight.

**Adaptation to Level-2 data.** Fabre and Challet [5] work with Level-3 data, where each order event is individually observed with its volume and distance of placement. Our dataset provides Level-2 snapshots (aggregate volume at each of $L = 10$ price levels), so the integrals in Equations (3.15)-(3.16) are approximated as follows. At each event $t$, the limit-order volume at level $i$ on side $s$ is inferred from positive volume changes at unchanged price levels: $[\Delta V^s_{i,t}]^+ \cdot \mathbf{1}[p^s_{i,t} = p^s_{i,t-1}]$. The distance mark $\delta^{L,s}_{i,t}$ is set to $c \cdot |p^s_{i,t} - p_{\text{mid},t}|$, where $c = 10^4$ converts Euro-denominated distances to a dimensionless quantity comparable to basis points. The scaling constant $c = 10^4$ was chosen empirically so that the typical distance of a Level-1 order ($\sim$ 0.01 EUR for TOTF.PA) maps to a dimensionless value of order $10^2$, placing it in a comparable range to the basis-point distances used by Fabre and Challet [5] on cryptocurrency data; $c$ is dataset-specific and should be recalibrated for instruments with a different tick size. Marketable-order flow is inferred from volume decreases at changing price levels. The continuous-time exponential decay $e^{-\beta t}$ is replaced by a discrete EWMA with half-life parameter $\beta$ (measured in events), applied to each side and kernel configuration independently.

**Parameter grid.** Following Fabre and Challet [5], the time scales are $\boldsymbol{\beta} = \{10, 100, 1{,}000\}$ and the distance scales are $\boldsymbol{\eta} = \{0.001, 0.1, 1.0, 10.0\}$ (expressed in $\text{bp}^{-1}$ in the original paper). These grid values follow Fabre and Challet [5], who select them on the basis of a multi-scale coverage argument without a formal ablation study; we adopt the same grid and do not perform an independent ablation in this work. With two sides of the book, $|\boldsymbol{\beta}| \times |\boldsymbol{\eta}| \times 2 = 24$ limit-order flow features form $\mathcal{L}_t$, and $|\boldsymbol{\beta}| \times 2 = 6$ marketable-order flow features form $\mathcal{M}_t$.

**Short- and long-horizon flows.** The half-lives $h_s = 5$ and $h_\ell = 50$ were chosen empirically to capture short-term (a few events) and medium-term (tens of events) order flow memory at the top of the book; no reference paper prescribes specific values, and a sensitivity analysis is deferred to Chapter 5. In addition to the kernel-weighted features above, limit-order and marketable-order flows at the best level (Level 1) are smoothed at two fixed half-lives ($h_s = 5$ and $h_\ell = 50$ events) for the limit component and at $h_s = 5$ for the market component, yielding six features: $L^b_t(h_s)$, $L^a_t(h_s)$, $M^b_t(h_s)$, $M^a_t(h_s)$, $L^b_t(h_\ell)$, $L^a_t(h_\ell)$. These correspond to the special case $\eta = 0$ (no spatial decay) with a single time scale, and capture the short- and long-term memory of order flow at the top of the book.

**Deep order insertions.** Volume increases at Level 5 are tracked separately to capture layering activity deep in the book. The positive volume change at Level 5, smoothed with an EWMA of half-life $h_d = 10$, produces two additional features (bid and ask). The half-life $h_d = 10$ and the choice of Level 5 as the monitoring depth were set empirically: Level 5 lies deep enough that large insertions there are unlikely to be filled, consistent with the layering signature described by Fabre and Challet [5], and $h_d = 10$ provides smoothing over a short horizon to reduce noise; sensitivity to both choices is deferred to Chapter 5.

### 3.1.7 Order Flow Imbalance

Order flow imbalance (OFI) measures the net signed change in supply and demand at each price level, conditioned on whether the price at that level has moved. Unlike the imbalance features of Section 3.1.1, which compare static snapshots of volume, OFI captures the directional pressure exerted by incoming orders.

At each level $i \in \{1, \ldots, 5\}$, the bid-side flow is:

$$\Delta F^b_{i,t} = \begin{cases} V^b_{i,t} & \text{if } p^b_{i,t} > p^b_{i,t-1}, \\ V^b_{i,t} - V^b_{i,t-1} & \text{if } p^b_{i,t} = p^b_{i,t-1}, \\ -V^b_{i,t-1} & \text{if } p^b_{i,t} < p^b_{i,t-1}, \end{cases} \tag{3.20}$$

and the ask-side flow $\Delta F^a_{i,t}$ follows the symmetric convention (positive when the ask price

improves, i.e. decreases). The OFI at level $i$ is

$$\mathrm{OFI}_{i,t} = \Delta F^b_{i,t} - \Delta F^a_{i,t}. \tag{3.21}$$

Positive values indicate net buying pressure; negative values indicate net selling pressure. Computing OFI at five separate levels allows the models to distinguish pressure at the best quotes from pressure deeper in the book, which is relevant for detecting layering.

### 3.1.8 Preprocessing and Scaling

Before the feature matrix enters any model, four cleaning steps are applied in sequence. The value $W_{\text{warmup}} = 3{,}000$ was chosen empirically to ensure that all rolling windows (up to $W = 100$) and EWMA features (with half-life up to $h_\ell = 50$) are fully populated before the output is consumed by any model; a sensitivity analysis is reported in Chapter 5. First, infinite values (arising from division by zero in rare edge cases) and `NaN`s are replaced by zero. Second, the initial $W_{\text{warmup}} = 3{,}000$ rows of each trading day are discarded to allow all EWMA and rolling-window features to stabilise. Third, each feature column is clipped at its 0.1% and 99.9% quantiles to remove extreme outliers while preserving the tail structure needed for anomaly detection. Fourth, any column with standard deviation below $10^{-9}$ is dropped as uninformative.

Three scaling methods are available, selected per experiment. *MinMax* scaling maps each feature to $[0, 1]$. *Standard* scaling centres each feature to zero mean and unit variance. The default method, following Fabre and Challet [5], is an empirical Box-Cox transformation. For each feature $j$, the Box-Cox transform with parameter $\lambda_j > 0$ is

$$T_{\lambda_j}(x^{(j)}) = \frac{(x^{(j)})^{\lambda_j} - 1}{\lambda_j}, \tag{3.22}$$

with the convention that $T_0(x) = \ln(x)$. The parameter $\lambda_j$ is chosen per feature to maximise the Gaussian log-likelihood of the transformed values on the training set. Before transformation, features with non-positive values are shifted by $|\min(x^{(j)})| + \epsilon$ (with $\epsilon = 10^{-6}$) to ensure strictly positive inputs; the shift is stored and applied consistently at inference time. After transformation, each feature is standardised to zero mean and unit variance using the training-set statistics:

$$z_{\lambda_j}(x^{(j)}) = \frac{T_{\lambda_j}(x^{(j)}) - \bar{T}_{\lambda_j}}{\sqrt{S_{\lambda_j}}}, \tag{3.23}$$

where $\bar{T}_{\lambda_j}$ and $S_{\lambda_j}$ are the empirical mean and variance of the transformed feature on the training set. This two-stage procedure addresses the heavy tails typical of LOB features, which can cause unstable weight updates during training. A legacy *Quantile* scaler (quantile transformation to a Gaussian followed by standard scaling) is retained for ablation purposes. In all cases, the scaler is fit on the training partition only and then applied to validation and test data, preventing information leakage across splits.

Table 3.1: Complete feature catalogue (∼85 features). All three models use the same feature set.

| Group | Representative features | Count | Description / Source |
|---|---|---|---|
| Imbalance | `L1_Imbalance`, `Weighted_Imbalance_*` | 5 | $(V^b - V^a)/(V^b + V^a)$ at L1, L5, and three weighted variants (Tao et al.) |
| Price & Spread Dynamics | `mid_price`, `spread`, `log_return`, `net_*_flow` | 13 | Mid-price, spread, log-return, volume deltas, velocity/acceleration, volatility, net flow |
| Elasticity | `ask_sweep_cost`, `bid_slope` | 4 | Sweep-to-fill cost (1000 units) and price-log(cumvol) regression slope per side |
| Volatility & Risk | `volatility_50`, `dt`, `log_dt` | 6 | Rolling stdev of returns (windows 50, 100), price range, abs. velocity, inter-event time |
| Event Flow / Rapidity | `size_trade_*`, `rapidity_cancel_*` | 18 | Poutré et al. trade/cancel sizes, SMA-10, rapidity (count/$\Delta t$), price speed |
| Hawkes Memory | `Hawkes_L_bid_beta*`, `Hawkes_L_ask_beta*`, `Deep_order_insertion_*` | 38 | Short/long EWMA of limit/market flow ($\beta \times \eta$ grid + fixed half-lives, Fabre & Challet), deep insertions |
| Order Flow Imbalance | `order_flow_imbalance_level_*` | 5 | Price-conditioned signed flow at levels 1–5 (Eq. (3.21)) |
| | | **89** | Total verified against processed feature files |

## 3.2 Detection Models

Three complementary unsupervised models are trained on the engineered feature set described in Section 3.1. Each model embodies a different inductive bias: the Transformer bottleneck autoencoder with One-Class SVM learns a compressed latent representation and fits a one-class boundary in that space; the probabilistic neural network predicts the full conditional distribution of the next mid-price move, providing both a likelihood-based anomaly score and an economic spoofing gain estimate; the probabilistic robust autoencoder learns to reconstruct normal patterns while explicitly down-weighting suspected anomalies during training. The diversity of approaches is intentional: reconstruction-based, density-estimation-based, and robust methods surface different signatures of anomalous order flow, and their agreement identifies the highest-confidence detections.

### 3.2.1 Transformer Bottleneck Autoencoder with One-Class SVM

This detector follows the two-stage framework of Poutré et al. [10]: a Transformer bottleneck autoencoder is trained to reconstruct LOB feature sequences, then its encoder is frozen and a One-Class SVM is fitted on the bottleneck representations. The anomaly score is a dissimilarity score derived from the OC-SVM decision function: for each input subsequence, the score measures how far the learned representation lies from the support of normal data (higher values indicate greater anomalousness). Converting this continuous score into a binary detection decision requires a threshold; the threshold selection procedure and its limitations are discussed in Section 3.3.

**Encoder.** The input to the encoder is a subsequence $\mathbf{S}_t \in \mathbb{R}^{k \times d_{\text{in}}}$, where $k$ is the sequence length and $d_{\text{in}}$ the number of features. A linear embedding projects each feature vector from $\mathbb{R}^{d_{\text{in}}}$ to

$\mathbb{R}^{d_{\text{model}}}$. The embedded sequence is scaled by $\sqrt{d_{\text{model}}}$ and summed with sinusoidal positional encodings following Vaswani et al. [14], providing the model with information about relative position within the subsequence. The encoded sequence passes through $N_{\text{enc}}$ Transformer encoder layers, each consisting of multi-head self-attention with $h$ heads followed by a position-wise feedforward network. After the final encoder layer, the output matrix $\mathbf{E}_t \in \mathbb{R}^{k \times d_{\text{model}}}$ is flattened to a vector $\mathbf{e}_t \in \mathbb{R}^{k \cdot d_{\text{model}}}$ and projected through a linear bottleneck to produce the representation:

$$\mathbf{z}_t = \mathbf{W}_b \, \mathbf{e}_t + \mathbf{b}_b \in \mathbb{R}^{d_{\text{rep}}}, \tag{3.24}$$

where $\mathbf{W}_b \in \mathbb{R}^{d_{\text{rep}} \times k \cdot d_{\text{model}}}$. Because $d_{\text{rep}} \ll k \cdot d_{\text{in}}$, the bottleneck forces the encoder to discard redundant information and retain only the features most useful for reconstruction. Anomalous subsequences, which deviate from the patterns seen during training, are expected to occupy regions of the latent space distinct from normal data.

**Decoder.** The decoder mirrors the encoder in reverse. A linear layer expands the bottleneck vector $\mathbf{z}_t$ back to $\mathbb{R}^{k \cdot d_{\text{model}}}$, which is reshaped to $\mathbb{R}^{k \times d_{\text{model}}}$. This matrix passes through $N_{\text{dec}}$ Transformer layers (using the same `TransformerEncoderLayer` architecture as the encoder, without cross-attention or causal masking) and a final linear projection from $\mathbb{R}^{d_{\text{model}}}$ to $\mathbb{R}^{d_{\text{in}}}$ produces the reconstruction $\hat{\mathbf{S}}_t$. Both the encoder and decoder use the same number of layers ($N_{\text{enc}} = N_{\text{dec}} = N$) and the same model dimension $d_{\text{model}}$, making the architecture symmetric.

**Training objective.** The autoencoder is trained to minimise the mean squared error between the input and its reconstruction:

$$\mathcal{L}_{\text{AE}}(\Theta) = \frac{1}{|\mathcal{D}|} \sum_{\mathbf{S}_t \in \mathcal{D}} \left\| \mathbf{S}_t - \hat{\mathbf{S}}_t \right\|_F^2, \tag{3.25}$$

where $\| \cdot \|_F$ denotes the Frobenius norm and $\mathcal{D}$ is the training set. The Adam optimiser [6] is used with a learning rate of $\eta = 10^{-4}$. Training proceeds for up to $E = 1{,}000$ epochs per day with early stopping on the validation loss (patience $P = 20$ epochs). Gradient norms are clipped to 1.0 to prevent instability.

**Architecture parameters.** The architecture follows Poutré et al. [10], who set the hyperparameters similarly to Vaswani et al. [14]: $h = 8$ attention heads, a feedforward dimension $d_{\text{ff}} = 4d_{\text{model}} = 512$, $N = 6$ Transformer encoder/decoder layers, and a representation dimension $d_{\text{rep}} = d_{\text{model}} = 128$. The sequence length ($k = 25$) also matches the original paper.

**One-Class SVM on bottleneck representations.** After the autoencoder is trained, the encoder is frozen and used to map every training subsequence to its bottleneck vector $\mathbf{z}_t \in \mathbb{R}^{d_{\text{rep}}}$. A One-Class SVM with a radial basis function (RBF) kernel is then fitted on these representations. The OC-SVM finds a hyperplane in the kernel-induced feature space $\mathcal{F}$ that separates the mapped data from the origin with maximum margin, solving:

$$\min_{\mathbf{w} \in \mathcal{F}, \xi, \rho} \quad \frac{1}{2}\|\mathbf{w}\|^2 - \rho + \frac{1}{\nu n} \sum_{i=1}^{n} \xi_i, \qquad \text{s.t.} \quad \mathbf{w}^\top \Phi(\mathbf{z}_i) \geq \rho - \xi_i, \quad \xi_i \geq 0, \tag{3.26}$$

where $\Phi$ is the feature map induced by the RBF kernel $k_\gamma(\mathbf{z}, \mathbf{z}') = \exp\!\left(-\gamma \|\mathbf{z} - \mathbf{z}'\|^2\right)$, $\nu \in (0, 1]$ bounds the fraction of training points allowed outside the boundary, and $\xi_i$ are slack variables.

The decision function for a test point $\mathbf{z}$ is:

$$f(\mathbf{z}) = \mathbf{w}^\top \Phi(\mathbf{z}) - \rho. \tag{3.27}$$

Positive values indicate inliers; negative values indicate outliers. Following the terminology of Poutré et al. [10] (Section 3.5), the *dissimilarity score* is defined as the negation of the decision function:

$$d(\mathbf{z}) = -f(\mathbf{z}) = \rho - \mathbf{w}^\top \Phi(\mathbf{z}). \tag{3.28}$$

Higher values indicate greater departure from the support of normal data, making $d(\mathbf{z})$ the anomaly score used throughout the pipeline. An event is classified as anomalous when $d(\mathbf{z}) \geq \tau$, where $\tau$ is a threshold determined by one of the methods in Section 3.3.

**Baseline threshold.** Because the EVT-based threshold methods (POT, SPOT, DSPOT) and the rolling FDR each require a calibration period, a simple *baseline threshold* is also defined for initial or comparative use:

$$\tau_0 = Q_{1-\epsilon}\big(\{d(\mathbf{z}_i)\}_{i \in \mathcal{D}_{\text{train}}}\big), \tag{3.29}$$

where $Q_p$ denotes the $p$-quantile and $\epsilon = 0.01$ is an assumed contamination rate matching the OC-SVM parameter $\nu$. This corresponds to flagging the top $\epsilon$ fraction of training scores, providing a label-free default that does not require an explicit threshold sweep.

The OC-SVM is applied to the bottleneck rather than to the raw features or to the reconstruction error for two reasons. First, the bottleneck compresses the $k \cdot d_{\text{in}} = 25 \times 80 = 2{,}000$ raw input dimensions down to $d_{\text{rep}} = 128$, producing a semantically compressed space where the OC-SVM can learn a tighter decision boundary. Second, Poutré et al. [10] show that the dissimilarity approach (OC-SVM on representations) substantially outperforms the reconstruction error alone, because normal data is more easily separable from anomalous data in the learned representation space than in the reconstruction error distribution.

The bandwidth parameter $\gamma$ is set via the median heuristic:

$$\gamma = \frac{1}{\text{median}\big(\|\mathbf{z}_i - \mathbf{z}_j\|^2\big)}, \tag{3.30}$$

computed over a random subsample of 2,000 pairs from the training representations. This differs from Poutré et al. [10], who use $\gamma = 10/(d_{\text{rep}} \cdot \text{Var}(\mathbf{z}))$; the median heuristic was found to produce better-calibrated kernel values in preliminary experiments on TOTF.PA data (the original formula yielded $\gamma$ values large enough to collapse $k_\gamma$ to near-zero between distinct points).

### Nyström Approximation and SGD Training

The Nyström kernel approximation does not appear in the paper of Poutré et al. [10], but it is the method used in their publicly available source code. The motivation is scalability: a standard kernel SVM requires computing and storing an $n \times n$ kernel matrix, which is infeasible when $n$ exceeds $10^5$ training subsequences. The Nyström method approximates the kernel matrix using a random subset of $m \ll n$ landmark points, reducing memory from $O(n^2)$ to $O(nm)$ and enabling GPU-accelerated training.

The approximation proceeds as follows. A set of $m$ landmark points $\mathbf{C} = \{\mathbf{c}_1, \ldots, \mathbf{c}_m\}$ is drawn uniformly at random from the training representations. The $m \times m$ kernel matrix $\mathbf{K}_{\text{CC}} = [k_\gamma(\mathbf{c}_i, \mathbf{c}_j)]_{i,j}$ is computed and regularised by adding $10^{-6}\mathbf{I}_m$. Its eigendecomposition $\mathbf{K}_{\text{CC}} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$ yields the normalisation matrix $\mathbf{U}\boldsymbol{\Lambda}^{-1/2}$. The approximate feature map for any point $\mathbf{z}$ is:

$$\tilde{\Phi}(\mathbf{z}) = \mathbf{k}(\mathbf{z}, \mathbf{C}) \cdot \mathbf{U}\boldsymbol{\Lambda}^{-1/2} \in \mathbb{R}^m, \tag{3.31}$$

where $\mathbf{k}(\mathbf{z}, \mathbf{C}) = \big[k_\gamma(\mathbf{z}, \mathbf{c}_1), \ldots, k_\gamma(\mathbf{z}, \mathbf{c}_m)\big]$.

After applying this feature map, the OC-SVM reduces to a linear problem in $\mathbb{R}^m$. The weight vector $\mathbf{w} \in \mathbb{R}^m$ and the offset $\rho$ are trained via stochastic gradient descent on the hinge-based objective:

$$\mathcal{L}_{\text{OC-SVM}}(\mathbf{w}, \rho) = \frac{1}{2}\|\mathbf{w}\|^2 - \rho + \frac{1}{\nu|\mathcal{B}|}\sum_{i \in \mathcal{B}}\big[-\big(\mathbf{w}^\top \tilde{\Phi}(\mathbf{z}_i) - \rho\big)\big]_+, \tag{3.32}$$

where $\mathcal{B}$ is a mini-batch of size 256 and $[\cdot]_+$ denotes the hinge function. SGD with momentum 0.9 and learning rate $10^{-2}$ runs for $E_{\text{SVM}} = 500$ epochs. After SGD converges, the offset $\rho$ is reset to the $\nu$-quantile of the training projections $\mathbf{w}^\top \tilde{\Phi}(\mathbf{z}_i)$, enforcing the KKT condition that exactly a fraction $\nu$ of training points lie outside the boundary regardless of SGD convergence quality.

The number of Nyström landmarks is set to $m = 300$; this value was chosen to balance approximation accuracy against memory cost on a single GPU, and was not tuned systematically.

Table 3.2: Hyperparameters of the Transformer bottleneck autoencoder and Nyström OC-SVM.

| **Parameter** | **Symbol** | **Value** | **Source** |
|---|---|---|---|
| Sequence length | $k$ | 25 | Poutré et al. [10] |
| Model dimension | $d_{\text{model}}$ | 128 | Poutré et al. [10] |
| Attention heads | $h$ | 8 | Poutré et al. [10] |
| Feedforward dimension | $d_{\text{ff}}$ | 512 | Poutré et al. [10] |
| Transformer layers | $N$ | 6 | Poutré et al. [10] |
| Representation dimension | $d_{\text{rep}}$ | 128 | Poutré et al. [10] |
| Training loss | — | MSE (Frobenius) | Poutré et al. [10] |
| Optimiser | — | Adam | Poutré et al. [10] |
| Learning rate | $\eta$ | $10^{-4}$ | Empirical |
| Max epochs | $E$ | 1,000 | Empirical |
| Early stopping patience | $P$ | 20 | Empirical |
| Gradient clipping | — | max norm 1.0 | Empirical |
| OC-SVM kernel | — | RBF | Poutré et al. [10] |
| Outlier fraction | $\nu$ | 0.01 | Empirical |
| RBF bandwidth | $\gamma$ | Median heuristic | Empirical |
| Nyström landmarks | $m$ | 300 | Empirical |
| SGD learning rate | — | $10^{-2}$ | Empirical |
| SGD momentum | — | 0.9 | Empirical |
| SGD epochs | $E_{\text{SVM}}$ | 500 | Empirical |
| SGD batch size | — | 256 | Empirical |

### 3.2.2 Probabilistic Neural Network

The PNN follows the framework of Fabre and Challet [5], who train a neural network to predict the conditional probability distribution of the next mid-price move given the current order flow state. Two quantities are derived from this prediction: the negative log-likelihood of the realised price move, which serves as a general anomaly score (events where the realised move is unlikely under the model receive high scores), and a spoofing gain that quantifies the profit a rational manipulator could extract from the current book state.

**Spoofing gain.** Before describing the network, we define the economic quantity that motivates it. Consider a manipulative agent who wants to sell $q$ units. She simultaneously places a bona fide ask order at distance $\delta_a$ from the best ask and a non-bona-fide (spoof) bid order of size $Q \gg q$ at distance $\delta_b$ from the best bid. The spoofing gain is the reduction in expected execution cost achieved by the spoof order:

$$G_t = \mathbb{E}\left[C^a_{\text{spoof}}(0, \delta_a, 0, q) \mid \mathbf{x}^0_t\right] - \mathbb{E}\left[C^a_{\text{spoof}}(\delta_b, \delta_a, Q, q) \mid \mathbf{x}_t\right], \tag{3.33}$$

where $C^a_{\text{spoof}}(\delta_b, \delta_a, Q, q)$ is the net cost of executing the strategy (genuine ask order filled, spoof bid order potentially filled, residual inventory liquidated at market) and $\mathbf{x}^0_t$, $\mathbf{x}_t$ denote the feature vectors without and with the spoof order, respectively [5]. $G_t > 0$ means spoofing is profitable at time $t$.

Under the assumptions of Fabre and Challet [5] (no partial fills, execution triggered when the mid-price crosses the order price, mandatory liquidation of unintended inventory), the expected cost depends on four terms involving the probability of fill on each side and the conditional expectation of the price move given fill or no fill. Let $s_t$ denote the bid-ask spread at time $t$, and let $\varepsilon^+$, $\varepsilon^-$ denote the maker and taker fees. For a seller, the expected cost is:

$$\begin{aligned}\mathbb{E}\left[C^a_{\text{spoof}}\right] = &-P\left(\Delta p > \delta_a + \tfrac{s_t}{2}\right)(1-\varepsilon^+)\, q\,(p^a_{1,t} + \delta_a)\\ &+ P\left(\Delta p < -\delta_b - \tfrac{s_t}{2}\right)(1+\varepsilon^+)\, Q\,(p^b_{1,t} - \delta_b)\\ &- P\left(\Delta p \le \delta_a + \tfrac{s_t}{2}\right)(1-\varepsilon^-)\, q\left(p^b_{1,t} + \mathbb{E}[\Delta p \mid \Delta p \le \delta_a + \tfrac{s_t}{2}]\right)\\ &- P\left(\Delta p < -\delta_b - \tfrac{s_t}{2}\right)(1-\varepsilon^-)\, Q\left(p^b_{1,t} + \mathbb{E}[\Delta p \mid \Delta p < -\delta_b - \tfrac{s_t}{2}]\right),\end{aligned} \tag{3.34}$$

where the probabilities and conditional expectations are evaluated under the PNN-predicted distribution of $\Delta p$.

**Architecture.** The PNN takes as input the feature vector $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ corresponding to the last time step of each sequence window, where $d_{\text{in}}$ is the number of features. This single-step architecture matches the design of Fabre and Challet [5], where the PNN operates as a memoryless (Markovian) predictor: it predicts the distribution of the next mid-price move from a single snapshot of the order book state. A single hidden layer with $d_h = 64$ units and ReLU activation maps the input to a hidden representation. A linear output layer produces three scalars $(\mu_{\text{raw}}, \sigma_{\text{raw}}, \alpha_{\text{raw}})$. The location parameter $\mu = \mu_{\text{raw}}$ and the skewness parameter $\alpha = \alpha_{\text{raw}}$ are unconstrained, while the scale parameter is made strictly positive by:

$$\sigma = \text{Softplus}(\sigma_{\text{raw}}) + \epsilon, \qquad \epsilon = 10^{-6}. \tag{3.35}$$

The output triple $(\mu, \sigma, \alpha)$ parameterises a skew-normal distribution for the next-step mid-price log return $y_t = \ln(p_{\text{mid},t+1}/p_{\text{mid},t})$.

**Loss function.** The training objective is the negative log-likelihood of the skew-normal distribution, following Equation (20) of Fabre and Challet [5]. The skew-normal density is:

$$f(y \mid \mu, \sigma, \alpha) = \frac{2}{\sigma}\,\varphi\left(\frac{y-\mu}{\sigma}\right)\Phi\left(\alpha \cdot \frac{y-\mu}{\sigma}\right), \tag{3.36}$$

where $\varphi$ is the standard normal density and $\Phi$ is the standard normal CDF. The per-batch loss is:

$$\mathcal{L}_{\text{PNN}} = -\frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} \ln\Big(f(y_i \mid \mu_i, \sigma_i, \alpha_i) + 10^{-10}\Big). \tag{3.37}$$

**Training procedure.** The Adam optimiser [6] is used with a learning rate of $\eta = 10^{-4}$ and a batch size of 64. Training runs for up to $E = 1{,}000$ epochs per day with early stopping on the validation loss (patience $P = 20$). The model is trained sequentially across trading days within the continual learning protocol described in Chapter 4.

**Anomaly scoring.** Two anomaly scores are available. The primary score, used for spoofing detection, is the spoofing gain $G_t$ defined in Equation (3.33): positive values flag events where spoofing would be profitable. The secondary score is the per-sample NLL $-\ln f(y_t \mid \mu_t, \sigma_t, \alpha_t)$: high values indicate that the realised price move was unlikely under the model, which may signal any form of anomalous activity, not only spoofing.

**Divergences from Fabre and Challet.** The following aspects of the implementation differ from the original paper.

1. **Data source.** Fabre and Challet [5] work with Level-3 cryptocurrency data (BTC-USD and ETH-USD on Coinbase), where each individual order is observed with its volume and distance of placement. Our dataset provides Level-2 equity snapshots (aggregate volume at 10 price levels for TOTF.PA, Euronext Paris). The Hawkes-inspired features are approximated from volume changes at unchanged price levels, as described in Section 3.1.6.

2. **Feature set.** The original paper uses 31 features (24 limit-order flow, 6 marketable-order flow, 1 spread). Our pipeline uses approximately 80 features from seven groups (Section 3.1), including imbalance, dynamics, elasticity, volatility, event flow, OFI, and Hawkes features. The PNN receives only the last time step of each sequence window, preserving the single-step Markovian architecture of Fabre and Challet.

3. **Batch size.** Fabre and Challet use a batch size of $2^{12} = 4{,}096$. Our implementation uses a batch size of 64.

4. **Learning rate and patience.** The paper uses a learning rate of $10^{-3}$ and early stopping patience of 100 epochs. Our implementation uses $10^{-4}$ and patience 20, tuned for the smaller and noisier equity dataset.

5. **Continual training.** The paper trains on a fixed dataset of $10^6$ orders. Our pipeline trains sequentially across trading days, carrying over weights from the previous day.

Table 3.3: Hyperparameters of the Probabilistic Neural Network.

| Parameter | Symbol | Value | Source |
|---|---|---|---|
| Input dimension | — | $d_{\text{in}}$ (last time step) | Fabre and Challet [5] |
| Hidden units | $d_h$ | 64 | Fabre and Challet [5] |
| Hidden activation | — | ReLU | Fabre and Challet [5] |
| Output distribution | — | Skew-normal $(\mu, \sigma, \alpha)$ | Fabre and Challet [5] |
| Loss | — | Skew-normal NLL | Fabre and Challet [5] |
| Optimiser | — | Adam | Fabre and Challet [5] |
| Learning rate | $\eta$ | $10^{-4}$ | Empirical |
| Batch size | $\lvert\mathcal{B}\rvert$ | 64 | Empirical |
| Max epochs | $E$ | 1,000 | Fabre and Challet [5] |
| Early stopping patience | $P$ | 20 | Empirical |

### 3.2.3 Probabilistic Robust Autoencoder

The PRAE, introduced by Lindenbaum et al. [7], addresses the contamination problem (Section 2.3) that affects all reconstruction-based anomaly detectors: when training data contains anomalous events, a standard autoencoder learns to reconstruct those anomalies alongside normal patterns, blurring the boundary between the two. The PRAE solves this by introducing a learnable per-sample gate that controls each training point's contribution to the reconstruction loss. During training, the gates are pushed toward 1 (inlier) by a regularisation term but toward 0 (outlier) by high reconstruction error, so the autoencoder focuses on reconstructing clean data.

**Backbone.** The PRAE uses the same bottleneck Transformer autoencoder architecture as the detector in Section 3.2.1 (same $d_{\text{model}}$, $h$, $N$, and $d_{\text{rep}}$). The Transformer backbone is wrapped

in the PRAE framework; the bottleneck representation is not extracted for a separate classifier but is used internally within the reconstruction pipeline.

**Stochastic gates.** Each training sample $i$ is assigned a learnable gate parameter $\mu_i \in \mathbb{R}$, initialised to 0.5. During each forward pass, a stochastic gate is drawn:

$$g_i = \text{clamp}(\mu_i + \varepsilon_i,\ 0,\ 1), \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \tag{3.38}$$

where $\sigma$ is a fixed noise scale. When $\mu_i$ drifts below zero, $g_i$ is clamped to zero with high probability, effectively removing sample $i$ from the reconstruction loss. When $\mu_i$ is large and positive, $g_i \approx 1$ and the sample contributes fully.

**Loss function.** Following the $\ell_1$ relaxation of Lindenbaum et al. [7] (their Equation 5), the training objective is:

$$\mathcal{L}_1 = \mathbb{E}\left(\sum_i g_i \|x_i - \hat{x}_i\|_2^2 - \lambda \|\mathbf{g}\|_1\right), \tag{3.39}$$

where the $\ell_1$ relaxation ($s = 1$) is used throughout this work. The expected gate value $\mathbb{E}[g_i]$ is computed analytically from the truncated Gaussian distribution of $g_i$ given $\mu_i$ and $\sigma$ (derived in the proof of Theorem 4.1 by Lindenbaum et al. [7]):

$$\mathbb{E}[g_i] = \frac{\sigma}{\sqrt{2\pi}}\left(e^{-\mu_i^2/(2\sigma^2)} - e^{-(1-\mu_i)^2/(2\sigma^2)}\right) + (\mu_i - 1)\,\Phi\left(\frac{1-\mu_i}{\sigma}\right) - \mu_i\,\Phi\left(\frac{-\mu_i}{\sigma}\right) + 1, \tag{3.40}$$

where $\Phi$ is the standard normal CDF. Using the analytical expectation rather than a Monte Carlo estimate provides smooth gradients for $\mu_i$ even when $g_i$ is clamped to 0 or 1, which stabilises training.

The regularisation parameter $\lambda > 0$ controls the trade-off between reconstruction fidelity and robustness. A large $\lambda$ pushes all gates toward 1, recovering a standard autoencoder. A small $\lambda$ allows the model to exclude more samples, increasing robustness but potentially discarding useful training signal.

Per-sample reconstruction errors are clamped to $10^6$ to prevent overflow: when a gate drifts to zero, the backbone receives no gradient for that sample, its reconstruction error can grow unboundedly, and the product $0 \cdot \infty$ produces NaN.

**Training.** The Adam optimiser with learning rate $\eta = 10^{-4}$ and batch size 64 is used, with early stopping on the validation loss (patience $P = 20$). On each new training day, the gate vector $\boldsymbol{\mu}$ is re-initialised to 0.5 for all samples in the current day's training set, because $\boldsymbol{\mu}$ has one entry per training sample and the sample count changes across days. The backbone weights are carried over from the previous day.

### Lambda Tuning

Lindenbaum et al. [7] propose two unsupervised schemes for tuning $\lambda$ (their Section 5.2). The first is a mean-energy heuristic: compute $\text{ME} = \frac{1}{N}\sum_{i=1}^{N} \|x_i\|_2^2$ and set $\lambda < \text{ME}$, since any $\lambda > \text{ME}$ would cause the model to include outliers. The second is validation-based: select the $\lambda$ that minimises reconstruction loss on held-out samples. In their experiments they set $\lambda = 1 < \text{ME}$; however, they note that a rigorous scheme for finding the optimal $\lambda$ remains an open question.

Our implementation combines both schemes. On the first training day, a mean-energy estimate $\lambda_{\text{heuristic}}$ is computed from a sample of training batches:

$$\lambda_{\text{heuristic}} = \overline{\|\mathbf{x}\|^2} \cdot k \cdot d_{\text{in}}, \tag{3.41}$$

where $\overline{\|\mathbf{x}\|^2}$ is the mean squared magnitude per element, and scaling by the sequence length $k$ and feature dimension $d_{\text{in}}$ recovers the mean total energy per sample (i.e. ME). A grid search is then run over the multiplicative factors $\{0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0\}$, training a fresh PRAE for 15 epochs for each candidate $\lambda = f \cdot \lambda_{\text{heuristic}}$. The factor that minimises the validation reconstruction loss (without gates) is selected, following the paper's second tuning scheme.

On subsequent training days (day 2 onward), the backbone is already partially trained, so the raw-energy heuristic overestimates the reconstruction scale. The selected $\lambda$ from day 1 is retained for all subsequent days without re-tuning.

**Anomaly scoring.** At inference, the stochastic gates are absent: the PRAE reduces to a standard autoencoder. The anomaly score for a test subsequence $\mathbf{S}_t$ is the per-sample squared reconstruction error:

$$s_t = \left\|\mathbf{S}_t - \hat{\mathbf{S}}_t\right\|^2. \tag{3.42}$$

High values indicate that the input deviates from the learned reconstruction manifold, which was fit to clean data thanks to the gating mechanism during training.

Table 3.4: Hyperparameters of the Probabilistic Robust Autoencoder.

| Parameter | Symbol | Value | Source |
|---|---|---|---|
| Backbone | — | Bottleneck Transformer (Table 3.2) | Empirical |
| Gate noise scale | $\sigma$ | 0.5 | Lindenbaum et al. [7] |
| Gate initialisation | $\mu_i$ | 0.5 | Empirical |
| $\lambda$ selection | — | Grid search (7 factors $\times$ 15 epochs) | Adapted from [7] |
| Reconstruction clamp | — | $10^6$ | Empirical |
| Optimiser | — | Adam | Empirical |
| Learning rate | $\eta$ | $10^{-4}$ | Empirical |
| Max epochs | $E$ | 1,000 | Empirical |
| Early stopping patience | $P$ | 20 | Empirical |
| Batch size | $\lvert\mathcal{B}\rvert$ | 64 | Empirical |

## 3.3 Anomaly Thresholding

This section formalises the four threshold methods surveyed conceptually in Section 2.4. Each of the three detection models produces a scalar anomaly score at every time step, but the relationship between that score and a detection decision differs fundamentally across models.

The probabilistic neural network of Fabre and Challet [5] outputs a full conditional distribution over future price moves, from which a spoofing gain $\Delta\mathcal{C}$ is computed at each time step. A positive gain indicates that a rational spoofer could profit from the current book state; the sign of the gain therefore serves as a natural detection threshold. More generally, the negative log-likelihood of the realised price move under the predicted distribution provides a probability-calibrated anomaly score whose magnitude can be interpreted directly. Thresholding is implicit in the model design: the economic framework supplies it.

The Transformer autoencoder of Poutré et al. [10] pairs its reconstruction with a One-Class SVM trained on the bottleneck representations. The OC-SVM produces a continuous dissimilarity score $d(\mathbf{z}) = -f(\mathbf{z})$ (Eq. 3.28) for each input subsequence, measuring how far the learned representation lies from the support of normal data. A threshold $\tau$ is then applied to this score to produce a binary detection decision. Crucially, $\tau$ is not learned during training and is not a direct output of the OC-SVM's own decision boundary (which is controlled by $\nu$). Instead, $\tau$

is selected post-hoc by sweeping over candidate values and choosing the one that maximises the $F_4$-score on a labelled evaluation set containing synthetically injected manipulations.

**Critical remark.** The $F_\beta$-score is defined as

$$F_\beta = (1+\beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}.$$

With $\beta = 4$, recall is weighted sixteen times more heavily than precision ($\beta^2 = 16$). This means the threshold is selected to minimise missed detections, at the cost of accepting a high false positive rate. For a market surveillance system that must act on every flagged event, the cost of false positives is not negligible: each flag triggers manual investigation, and a high false positive rate degrades analyst efficiency and trust in the system.

Optimising $\tau$ against an $F_4$-score requires ground-truth manipulation labels. In real deployment settings, such labels are either unavailable (equity markets with no public order-level annotations) or unreliable (cryptocurrency markets where manipulation is not prohibited and thus not labelled). The procedure is therefore not directly applicable to the unsupervised detection setting that motivates this project.

Furthermore, if $\tau$ is selected by sweeping on the same labelled set used to report detection results, the reported performance is optimistic: the threshold has been tuned to the evaluation data. Poutré et al. [10] select $\tau$ by maximising the $F_4$-score on the test set containing their synthetic manipulations, and then report detection metrics on that same test set. This is a methodological concern that should be noted when interpreting any comparison against their reported numbers.

The Probabilistic Robust Autoencoder (PRAE) of Lindenbaum et al. [7] learns per-sample gates during training that downweight suspected anomalies, but these gates are not available for new, unseen test data. At inference time, the only available signal is the reconstruction score (e.g. the negative log-likelihood or the squared reconstruction error). The paper does not specify a procedure for converting this score into a binary detection decision: no threshold selection method is proposed.

Both the Transformer+OC-SVM and the PRAE therefore lack a principled, label-free threshold selection procedure. The three methods below address this gap. POT and SPOT/DSPOT provide statistically principled false positive rate control under a Generalised Pareto tail model; the rolling FDR controls the expected proportion of false detections in a streaming setting. These methods select thresholds using only the statistical properties of the score distribution, without access to labels, which is precisely the setting of this project. They may also serve as drop-in replacements for the spoofing gain threshold in the PNN, decoupling the scoring and thresholding components and allowing a uniform evaluation framework across all three models. Throughout, $s_t$ denotes the anomaly score at time $t$, $\tau_t$ denotes the (possibly time-varying) threshold, and a detection is triggered when $s_t > \tau_t$, producing the binary indicator $\mathbf{1}[s_t > \tau_t]$.

### 3.3.1 Peak Over Threshold

**Theoretical foundation.** The Pickands-Balkema-de Haan theorem [1, 9] guarantees that, for a wide class of distributions (specifically, distributions whose maximum belongs to one of the three max-stable families), the excess of a random variable above a sufficiently high threshold $u$ converges in distribution to a Generalised Pareto Distribution (GPD). Concretely, if $Y = X - u \mid X > u$, then for large $u$:

$$F_{\xi,\sigma}(y) = 1 - \left(1 + \xi \frac{y}{\sigma}\right)^{-1/\xi}, \qquad y > 0, \quad 1 + \xi \frac{y}{\sigma} > 0, \tag{3.43}$$

where $\xi \in \mathbb{R}$ is the shape (extreme value index) and $\sigma > 0$ is the scale. When $\xi > 0$ the tail is heavy (Fréchet domain); when $\xi < 0$ the tail is bounded (Weibull domain). In the limiting case $\xi = 0$ the GPD reduces to the exponential distribution $F_{0,\sigma}(y) = 1 - \mathrm{e}^{-y/\sigma}$.

**Parameter estimation.** The parameters $(\xi, \sigma)$ are estimated by maximum likelihood from the empirical exceedances above $u$. The log-likelihood for $N_u$ observed exceedances $y_1, \ldots, y_{N_u}$ is:

$$\ell(\xi, \sigma) = -N_u \ln \sigma - \left(1 + \frac{1}{\xi}\right) \sum_{i=1}^{N_u} \ln\left(1 + \frac{\xi}{\sigma} y_i\right). \tag{3.44}$$

Grimshaw's trick [11] reduces this two-variable optimisation to a single-variable root search. Defining $x = \xi/\sigma$ and the auxiliary functions $u(x) = \frac{1}{N_u}\sum_i (1 + x\,y_i)^{-1}$ and $v(x) = 1 + \frac{1}{N_u}\sum_i \ln(1 + x\,y_i)$, the MLE satisfies $u(x)\,v(x) = 1$, from which $\hat{\xi} = v(x^*) - 1$ and $\hat{\sigma} = \hat{\xi}/x^*$. The implementation searches for roots of $w(x) = u(x)\,v(x) - 1$ in the two intervals $\left(-1/y_{\max} + \epsilon,\ -\epsilon\right)$ and $\left[2(\bar{y} - y_{\min})/(\bar{y}\,y_{\min}),\ 2(\bar{y} - y_{\min})/y_{\min}^2\right]$ using L-BFGS-B minimisation of $\sum_k w(x_k)^2$ with $k = 10$ candidate starting points, following the procedure of Siffer et al. [11]. The candidate with the highest log-likelihood is retained, including the $\xi = 0$ case ($\hat{\sigma} = \bar{y}$).

**Initial threshold.** The initial threshold $u$ is set to the $q_0$-th empirical quantile of the score distribution, with $q_0 = 0.98$ as recommended by Siffer et al. [11]: "In practice we set $t$ to a high empirical quantile (98%)." If no exceedances exist above $u$ (i.e. $N_u = 0$), the implementation falls back to $\tau = u$.

**Threshold derivation.** Given fitted $(\hat{\xi}, \hat{\sigma})$ and a target false positive rate $q$, the detection threshold is:

$$\tau = u + \frac{\hat{\sigma}}{\hat{\xi}}\left[\left(\frac{q\,n}{N_u}\right)^{-\hat{\xi}} - 1\right], \qquad \hat{\xi} \neq 0, \tag{3.45}$$

where $n$ is the total number of observations and $N_u$ is the number of exceedances above $u$. When $\hat{\xi} = 0$, the threshold reduces to $\tau = u - \hat{\sigma}\,\ln(q\,n/N_u)$. This formula follows directly from inverting the GPD survival function at probability $q \cdot n/N_u$ and is Equation 1 of Siffer et al. [11].

**Static application.** POT is a batch method: it fits the GPD on a fixed set of scores and produces a single threshold that does not change at inference time. This is appropriate when the score distribution is stationary across the calibration and test periods, but it cannot adapt to non-stationarity.

### 3.3.2 SPOT and DSPOT

**Implementation note.** The SPOT and DSPOT implementations used in this project are taken directly from the source code released by Siffer et al. [11] alongside their paper. They were not reimplemented from scratch. As a result, any deviation between the code and the algorithmic descriptions in Section 3.3.2 reflects choices made in Siffer et al.'s own implementation, not modifications introduced in this project. The descriptions below follow the code as released; where the code and the paper differ, both are noted.

**SPOT (Streaming Peak Over Threshold).** SPOT [11] extends POT to a streaming setting by maintaining a running tail model and updating the threshold as new scores arrive. The algorithm proceeds as follows:

1. **Initialisation.** Run the POT procedure on the first $n_{\text{init}}$ scores to obtain an initial threshold $\tau_0$ and an initial peak threshold $u$. Collect the initial exceedances $\mathcal{Y} = \{s_i - u \mid s_i > u,\ i \leq n_{\text{init}}\}$.

2. **Streaming update.** For each subsequent score $s_t$:
   - If $s_t > \tau_{t-1}$: flag $s_t$ as an anomaly. The peak set $\mathcal{Y}$ and $\tau$ are not updated.
   - If $u < s_t \leq \tau_{t-1}$: add the exceedance $s_t - u$ to $\mathcal{Y}$, refit $(\hat{\xi}, \hat{\sigma})$ via Grimshaw, and recompute $\tau_t$ using Equation (3.45).
   - If $s_t \leq u$: no update.

The GPD is thus refit after every new peak, making the threshold progressively more accurate as more tail observations accumulate. The running observation count $k$ in the denominator of Equation (3.45) is incremented for every non-anomalous observation, so the threshold accounts for the growing sample size.

**Implementation note.** In the implementation, the initial POT fit is computed on the full score array rather than on only the first $n_{\text{init}}$ observations. This means that the initial peak threshold $u$ is the 98th percentile of all scores, not just the calibration batch. The practical effect is a more stable $u$, but it grants the initialisation access to future information. The initial peak set $\mathcal{Y}$ is correctly restricted to the first $n_{\text{init}}$ observations. Additionally, the running counter $N$ in the threshold formula counts all observations (including flagged anomalies), whereas Algorithm 2 of Siffer et al. increments $k$ only for non-anomalous observations. This makes the ratio $qN/N_u$ slightly larger, producing a marginally higher (more conservative) threshold.

**DSPOT (Drift SPOT).** DSPOT [11] extends SPOT to non-stationary streams by applying the peak-over-threshold logic to drift-corrected scores rather than raw scores. The drift correction subtracts a local baseline $M_t$ estimated from a sliding window of the $d$ most recent normal (non-anomalous) observations:

$$s'_t = s_t - M_t, \qquad M_t = \frac{1}{d} \sum_{k=1}^{d} s^*_{t-k}, \tag{3.46}$$

where $s^*_{t-1}, \ldots, s^*_{t-d}$ are the last $d$ non-anomalous scores. The drift-corrected scores $s'_t$ are assumed to come from a stationary distribution, and SPOT is run on this transformed stream. The effective detection threshold at time $t$ is $\tau_t = \tau'_t + M_t$, where $\tau'_t$ is the SPOT threshold in the drift-corrected space.

When $s_t$ is flagged as an anomaly, the window $W^*$ is not updated and $M_{t+1} = M_t$, preventing anomalous values from contaminating the baseline estimate. In peak and normal cases, the oldest value in $W^*$ is replaced by $s_t$ and $M_{t+1}$ is recomputed. This matches Algorithm 3 of Siffer et al.

**Implementation note.** In the initialisation loop of DSPOT, the local mean is computed from $d-1$ values instead of $d$ due to an off-by-one in the slice indices. In the streaming phase, the window correctly maintains $d$ values. The effect is negligible for $d \gg 1$. The same counter divergence as in SPOT applies: $N$ counts all observations rather than only non-anomalous ones.

**Shared parameters.** Table 3.5 lists the parameters shared by POT, SPOT, and DSPOT.

Table 3.5: Parameters of the EVT-based threshold methods.

| Parameter | Symbol | Value | Source |
|---|---|---|---|
| Risk level | $q$ | $10^{-3}$ | Siffer et al. [11] |
| Initial quantile level | $q_0$ | 0.98 | Siffer et al. [11] |
| Grimshaw candidates | $k$ | 10 | Siffer et al. [11] |
| Numerical tolerance | $\epsilon$ | $10^{-8}$ | Empirical |
| SPOT/DSPOT init size | $n_{\text{init}}$ | 1,000 | Siffer et al. [11] |
| DSPOT window depth | $d$ | 200 | Siffer et al. [11] |

### 3.3.3 Rolling False Discovery Rate

**FDR control background.** The Benjamini-Hochberg (BH) procedure [3] controls the false discovery rate (FDR), defined as the expected proportion of false positives among all rejected hypotheses. Given $m$ hypothesis tests with p-values $p_{(1)} \leq \cdots \leq p_{(m)}$, the BH procedure rejects all hypotheses $H_{(i)}$ for which

$$p_{(i)} \leq \frac{i}{m}\alpha, \tag{3.47}$$

where $\alpha$ is the target FDR level. Under independence (or positive regression dependence) of the p-values, this guarantees FDR $\leq \alpha$.

**Adaptation to anomaly scores.** Translating anomaly scores to p-values requires a null distribution for the scores. The implementation constructs this null from the rolling window itself, using a log-transform followed by robust z-scoring. Given a window of scores $\{s_{t-W+1}, \ldots, s_t\}$:

1. **Log-transform.** Compute $\tilde{s}_i = \ln(s_i + c)$, where $c = |\min_j s_j| + 10^{-6}$ if any score is non-positive, and $c = 0$ otherwise. The log-transform reduces the skewness of heavy-tailed score distributions.

2. **Robust z-score.** Compute the median $\tilde{\mu} = \text{med}(\tilde{s})$ and the median absolute deviation $\text{MAD} = \text{med}(|\tilde{s} - \tilde{\mu}|)$, then
$$z_i = \frac{\tilde{s}_i - \tilde{\mu}}{1.4826 \cdot \text{MAD}}, \tag{3.48}$$
where the factor 1.4826 is the consistency constant that makes $1.4826 \cdot \text{MAD}$ an unbiased estimator of the standard deviation under normality. If $\text{MAD} = 0$, it is replaced by the mean absolute deviation plus $10^{-10}$.

3. **P-values.** Under the assumption that the null scores follow a standard normal distribution in the transformed space, the one-sided p-value is $p_i = 1 - \Phi(z_i)$, where $\Phi$ is the standard normal CDF.

4. **BH rejection.** Sort the $W$ p-values in ascending order and apply Equation (3.47). The score $s_t$ is flagged as anomalous if its p-value is among the rejected set. The dynamic threshold is the raw score corresponding to the largest rejected p-value.

No decisions are made until the window contains at least 20 observations, ensuring a minimum sample size for stable z-score estimation.

**Rolling window mechanics.** The window is a first-in, first-out buffer of fixed length $W$. At each time step, the newest score enters the window, the oldest is discarded, and the full BH procedure is reapplied. The null distribution is therefore re-estimated at every step from the local window, allowing RFDR to track non-stationarity without an explicit drift model. A shorter window responds faster to local anomaly clusters but has lower statistical power for isolated anomalies; a longer window has higher power but lags behind distributional shifts.

No adaptive $\hat{\pi}_0$ estimator (e.g. Storey's method) is applied; the standard BH procedure is used throughout.

**Design rationale.** The RFDR method is not drawn from a single reference paper. It adapts the classical BH procedure [3] to the streaming anomaly detection setting by combining it with rolling-window null estimation and robust score normalisation. This combination is a contribution of the present project.

**Parameters.** Table 3.6 lists the RFDR parameters.

Table 3.6: Parameters of the Rolling False Discovery Rate method.

| Parameter | Symbol | Value | Source |
|---|---|---|---|
| FDR level | $\alpha$ | 0.05 | Empirical |
| Rolling window length | $W$ | 500 | Empirical |
| Minimum history | — | 20 | Empirical |
| MAD consistency constant | — | 1.4826 | Exact under normality |

## 3.4 Spoofing Gain Estimation

The spoofing gain is the central quantity that connects the PNN's distributional predictions (Section 3.2.2) to the detection of manipulative intent. It serves two roles in the pipeline: it defines the detection signal for the PNN model (a positive gain flags a book state where spoofing would be profitable), and it provides an economically interpretable complement to the statistical anomaly scores produced by the Transformer+OC-SVM and PRAE models. Note that the spoofing gain framework targets layering and spoofing strategies specifically; other forms of manipulation such as quote stuffing, which operate through message-rate flooding rather than price manipulation, are captured instead by the rapidity and event-flow features (Section 3.1.5) and flagged by the Transformer+OC-SVM and PRAE anomaly scores. This section consolidates the formal framework introduced in Section 3.2.2, describes how the gain is computed in practice, and documents the parameter choices and edge-case handling in the implementation.

**Strategy and assumptions.** The framework of Fabre and Challet [5] models a manipulative agent who wants to execute a genuine trade of size $q$ on one side of the book. To improve her execution price, she simultaneously places a non-bona-fide order of size $Q \gg q$ on the opposite side at distance $\delta_b$ from the best quote, inflating the perceived liquidity and shifting the predicted mid-price distribution in her favour. Three assumptions govern the model. First, no partial fills occur: at the trading horizon, each order is either completely filled or untouched. Second, execution is triggered by a first-passage criterion: the bid order fills if $\Delta p < -(\delta_b + s_t/2)$ and the ask order fills if $\Delta p > \delta_a + s_t/2$, where $\Delta p$ is the mid-price move over the horizon and $s_t$ is the bid-ask spread. Third, the agent must execute her target quantity within the horizon: if the genuine order is not filled, she crosses the spread at a taker fee $\varepsilon^-$; if the spoof order is filled inadvertently, she liquidates the resulting inventory at market.

**Gain definition.** For a seller, the spoofing gain $G_t$ is the reduction in expected execution cost achieved by the spoof order (Equation (3.33)):

$$G_t = \underbrace{\mathbb{E}\big[C^a_{\text{spoof}}(0, \delta_a, 0, q) \mid \mathbf{x}^0_t\big]}_{\text{cost without spoofing}} - \underbrace{\mathbb{E}\big[C^a_{\text{spoof}}(\delta_b, \delta_a, Q, q) \mid \mathbf{x}_t\big]}_{\text{cost with spoofing}},$$

where $\mathbf{x}^0_t$ and $\mathbf{x}_t$ denote the feature vectors without and with the spoof order. The cost function $C^a_{\text{spoof}}$ decomposes into four terms (Equation (3.34)): (1) revenue from the filled genuine ask

order, weighted by the maker fee $\varepsilon^+$; (2) cost of the inadvertently filled spoof bid order; (3) cost of liquidating the unfilled genuine order by crossing the spread; and (4) cost of liquidating the filled spoof order's inventory at market. Each term involves a fill probability $P(\Delta p \gtrless \tau)$ and a conditional expectation $\mathbb{E}[\Delta p \mid \Delta p \gtrless \tau]$, where the threshold $\tau$ depends on the order distance and the spread. These quantities are evaluated analytically from the PNN-predicted skew-normal parameters $(\mu_t, \sigma_t, \alpha_t)$ using the closed-form expressions for the skew-normal CDF $F_\alpha$ and truncated first moments (Equations 35–37 of Fabre and Challet [5]):

$$P(\Delta p \leq x) = F_\alpha\left(\frac{x-\mu}{\sigma}\right), \tag{3.49}$$

$$\mathbb{E}[\Delta p \mid \Delta p \leq x] = \mu + \sigma \frac{\sqrt{2/\pi}\,\beta\,\Phi\left(\sqrt{1+\alpha^2}\,\frac{x-\mu}{\sigma}\right) - e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}\Phi\left(\alpha\,\frac{x-\mu}{\sigma}\right)}{F_\alpha\left(\frac{x-\mu}{\sigma}\right)}, \tag{3.50}$$

where $\beta = \alpha/\sqrt{1+\alpha^2}$ and $\Phi$ is the standard normal CDF. The conditional expectation $\mathbb{E}[\Delta p \mid \Delta p > x]$ follows by symmetry. This makes $G_t$ fully computable from a single order book snapshot and the PNN output, with no simulation or numerical integration required.

The framework is symmetric for a buyer: she places the genuine order on the bid side at distance $\delta_b$ and the spoof order on the ask side at distance $\delta_a$, with the corresponding cost function $C^b_{\text{spoof}}$ defined in Equation (28) of Fabre and Challet [5]. In this project, the gain is evaluated from the ask side (seller perspective); the bid-side gain follows by the same construction with reversed roles.

**Detection rule.** $G_t > 0$ indicates that spoofing is, in expectation, profitable at time $t$: the cost reduction from the artificially induced price move exceeds the risk of the spoof order being filled. Fabre and Challet [5] use this condition directly as their detection rule, labelling as suspicious all large orders whose size exceeds a filtering threshold of $Q \geq 4{,}500$ USD for which $G_t > 0$. In our implementation, $Q = 4{,}500$ is used as a fixed spoof order size parameter rather than a filtering threshold, because our Level-2 data does not record individual order sizes. There is no transformation of the raw gain into a probability or score in $[0, 1]$: the gain is a signed real-valued signal whose magnitude reflects the expected profitability of the hypothetical spoof.

**Computation pipeline.** The PNN is not trained on spoofing gain labels. Its training objective is the negative log-likelihood of the skew-normal distribution (Equation (3.37)), which teaches the network to predict the conditional distribution of mid-price moves from order flow features. The gain is computed post-hoc: for each time step $t$, the PNN predicts $(\mu_t, \sigma_t, \alpha_t)$, and the cost function (3.34) is evaluated twice, once with spoofing parameters $(Q, \delta_b)$ and once with $(Q{=}0, \delta_b{=}0)$. The difference yields $G_t$.

Table 3.7 lists the parameters used in the gain computation. The values of $Q$, $q$, and $\delta_a$ follow Fabre and Challet [5]. The fee parameters $\varepsilon^+$ and $\varepsilon^-$ are recalibrated from the original Coinbase cryptocurrency fee schedule ($\varepsilon^+ = 0$, $\varepsilon^- = 0.05$) to values representative of Euronext Paris large-cap equity trading ($\varepsilon^+ = 0$, $\varepsilon^- = 8 \times 10^{-4}$), which reduce the taker fee by approximately two orders of magnitude and yield spoofing gain estimates that are economically meaningful for the equity venue. The spoof distance $\delta_b = 0.01$ places the non-bona-fide order roughly 100 basis points from the best bid, matching the deepest distance in the sensitivity grid of Fabre and Challet (who vary $\delta$ over $\{0, 1, 10, 100\}$ bps). All parameters are fixed across the entire evaluation period; none are estimated from data.

Table 3.7: Parameters of the spoofing gain computation.

| Parameter | Symbol | Value | Source |
|---|---|---|---|
| Minimum Spoof order size | $Q$ | 4,500 | Empirical (adapted from [5]) |
| Genuine order size | $q$ | 100 | Fabre and Challet [5] |
| Genuine order distance | $\delta_a$ | 0 | Fabre and Challet [5] |
| Spoof order distance | $\delta_b$ | 0.01 | Empirical |
| Maker fee | $\varepsilon^+$ | 0 | Euronext Paris (large-cap equities) |
| Taker fee | $\varepsilon^-$ | $8 \times 10^{-4}$ | Euronext Paris (large-cap equities) |

**Edge cases.** Several degenerate configurations require special handling. When the bid-ask spread is zero or negative (a crossed book), the fill thresholds $\delta_a + s_t/2$ and $\delta_b + s_t/2$ collapse, making the cost function numerically unstable. The implementation floors the spread at $10^{-4}$ to prevent this. When the predicted scale $\sigma_t$ is near zero, the skew-normal CDF evaluations become degenerate; the Softplus activation in the PNN output layer (Equation (3.35)) prevents $\sigma_t$ from reaching exactly zero, and the conditional expectation functions add $10^{-10}$ to CDF denominators to guard against residual underflow. No clipping is applied to the computed gain: extreme positive or negative values are retained. The paper does not discuss any of these edge cases.

**Implementation note.** The full gain computation of Equation (3.33) requires two forward passes through the PNN: one with the feature vector $\mathbf{x}_t$ containing the observed order and one with the counterfactual vector $\mathbf{x}_t^0$ obtained by removing the order's contribution to the Hawkes features. This is feasible when individual order insertions are observed (Level-3 data), but not with our Level-2 snapshots, which record only aggregate volume at each price level. Accordingly, the batch implementation evaluates the cost-function counterfactual using a single set of PNN-predicted parameters $(\mu_t, \sigma_t, \alpha_t)$: the no-spoof cost sets $Q = 0$ and $\delta_b = 0$, while the with-spoof cost uses the actual parameter values. This batch path is used in all experiments because the Level-2 data does not permit constructing the counterfactual feature vector $\mathbf{x}_t^0$ required by the two-forward-pass path (which exists in the code but is designed for Level-3 data). The batch path captures the mechanical effect of different fill thresholds but does not capture the distributional shift that the spoof order would induce in the Hawkes features and hence in the predicted $(\mu_t, \sigma_t, \alpha_t)$. The result is a conservative estimate of the true gain, because the price-impact channel through which the spoof order shifts $\mu_t$ in the spoofer's favour is absent.

**Relationship to thresholding.** The spoofing gain provides a natural, economically motivated threshold: $G_t > 0$ defines the boundary between book states where spoofing is profitable and states where it is not. For the PNN, this threshold is implicit in the model's output and requires no additional calibration. The EVT-based (POT, SPOT, DSPOT) and rolling FDR methods described in Section 3.3 can alternatively be applied to the raw gain series, treating it as an anomaly score and selecting a threshold that controls the tail probability or false discovery rate. This enables a uniform evaluation framework across all three models, where the threshold is decoupled from the scoring function.

## 3.5 Interpretability

Each of the three detection models produces a scalar anomaly score per time step (Section 3.2). A score alone is not actionable: a market surveillance analyst needs to know which features of the order book drove a high score before deciding whether to escalate. Two complementary attribution methods are used. Integrated Gradients [12] provides a fine-grained, gradient-based attribution at the level of individual features (and individual time steps within the sequence

window). Grouped Occlusion provides a coarser but more robust attribution at the level of feature groups, which maps directly to the domain concepts introduced in Section 3.1: imbalance, volatility, Hawkes flow, OFI, dynamics, and so on.

### 3.5.1 Integrated Gradients

**Motivation.** The simplest gradient-based attribution assigns to each input feature $x_i$ the product $x_i \cdot \partial f / \partial x_i$ evaluated at the input point. This approach fails when the model saturates: if the prediction function flattens at the input, the gradient is near zero even for features that were essential to the prediction. Sundararajan et al. [12] demonstrate this failure concretely for ReLU networks and propose Integrated Gradients (IG) as a principled alternative that integrates gradients along a straight-line path from a baseline input $\mathbf{x}'$ to the actual input $\mathbf{x}$, eliminating the dependence on the local gradient at a single point.

**Formal definition.** Let $f : \mathbb{R}^d \to \mathbb{R}$ denote the scalar target function whose output is to be attributed, $\mathbf{x} \in \mathbb{R}^d$ the input, and $\mathbf{x}' \in \mathbb{R}^d$ the baseline. The integrated gradient for feature $i$ is:

$$\mathrm{IG}_i(\mathbf{x}) = (x_i - x_i') \int_0^1 \frac{\partial f\big(\mathbf{x}' + \tau(\mathbf{x} - \mathbf{x}')\big)}{\partial x_i} \, d\tau. \tag{3.51}$$

Two axioms established by Sundararajan et al. distinguish IG from alternative methods.

*Completeness.* The attributions sum exactly to the model-output difference:

$$\sum_{i=1}^{d} \mathrm{IG}_i(\mathbf{x}) = f(\mathbf{x}) - f(\mathbf{x}'). \tag{3.52}$$

No attribution is lost or fabricated; every unit of score change is accounted for by the input features.

*Sensitivity.* If $\mathbf{x}$ and $\mathbf{x}'$ differ in exactly one feature $i$ and $f(\mathbf{x}) \neq f(\mathbf{x}')$, then $\mathrm{IG}_i(\mathbf{x}) \neq 0$. This rules out the saturation failure mode of plain gradients, where a feature can receive zero attribution despite being the sole cause of a prediction change.

Sundararajan et al. further prove that IG is the unique symmetry-preserving path method satisfying completeness, sensitivity, implementation invariance, and linearity (Proposition 2 and Theorem 1 of [12]).

**Approximation.** The integral in Equation (3.51) is approximated by a Riemann sum over $m$ uniformly spaced interpolation steps:

$$\mathrm{IG}_i(\mathbf{x}) \approx (x_i - x_i') \cdot \frac{1}{m} \sum_{k=1}^{m} \frac{\partial f\big(\mathbf{x}' + \frac{k}{m}(\mathbf{x} - \mathbf{x}')\big)}{\partial x_i}. \tag{3.53}$$

Each step requires one forward and one backward pass through the model, so computation scales linearly in $m$. The implementation uses the trapezoidal rule (averaging adjacent gradient evaluations) for improved accuracy. The number of steps is set to $m = 50$, matching the recommendation in Sundararajan et al. [12], who report that 20 to 300 steps suffice for convergence in practice.

**Baseline choice.** The baseline $\mathbf{x}'$ represents the "absence of signal" from which attributions are measured. The implementation uses the zero vector $\mathbf{x}' = \mathbf{0}$. Because the input features are standardised to zero mean and unit variance (Section 3.1), the zero baseline corresponds to the average market state across the training set, which is a natural reference point: attributions then measure how much each feature's deviation from its mean contributes to the anomaly score.

**Application in this project.** IG is applied to two models. For the PNN (Section 3.2.2), the target function is the predicted scale parameter: $f(\mathbf{x}) = \sigma(\mathbf{x})$. High predicted volatility is a necessary condition for large spoofing gains, so attributing $\sigma$ identifies which features inflate the model's uncertainty. For the PRAE (Section 3.2.3), the target function is the reconstruction error: $f(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$, which is the anomaly score itself.

IG is not applied to the Transformer+OC-SVM detector, because the OC-SVM decision function is a non-parametric kernel machine fitted to latent representations and gradients cannot flow from the OC-SVM score back through the encoder in a single differentiable computation graph. Grouped Occlusion (Section 3.5.2) addresses this gap.

Table 3.8: Integrated Gradients parameters.

| Parameter | Symbol | Value | Source |
|---|---|---|---|
| Interpolation steps | $m$ | 50 | Sundararajan et al. [12] |
| Baseline | $\mathbf{x}'$ | $\mathbf{0}$ | Standard (zero mean after standardisation) |
| Integration rule | — | Trapezoidal | Empirical |
| Target (PNN) | $f$ | $\sigma(\mathbf{x})$ | Empirical |
| Target (PRAE) | $f$ | $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$ | Empirical |

### 3.5.2 Grouped Occlusion

Grouped Occlusion is a project-specific attribution method designed for the Transformer+OC-SVM detector, where gradient-based attribution is not applicable. It extends the standard single-feature occlusion paradigm by operating at the level of semantically defined feature groups, reducing noise from correlated features and producing attributions that map directly to the domain concepts of Section 3.1.

**Definition.** Let the input $\mathbf{x} \in \mathbb{R}^{T \times d}$ (a sequence of $T$ time steps with $d$ features) be partitioned into $K$ non-overlapping groups $\mathcal{G}_1, \ldots, \mathcal{G}_K$, where each group corresponds to a feature family (e.g., all imbalance features, all Hawkes features), a side of the book (bid, ask, or neutral), a price level, or a combination thereof. Let $\mathbf{x}^{(\setminus k)}$ denote the input with every feature in group $\mathcal{G}_k$ replaced by its baseline value across all time steps. The occlusion attribution for group $k$ is:

$$\mathrm{Occ}_k(\mathbf{x}) = f(\mathbf{x}) - f(\mathbf{x}^{(\setminus k)}), \tag{3.54}$$

where $f$ is the anomaly score function (the negated OC-SVM decision function). A positive $\mathrm{Occ}_k$ indicates that group $k$ contributes to a higher anomaly score; removing it reduces the score. A negative value indicates that the group suppresses the score.

**Grouping criteria.** The implementation supports several grouping schemes: by feature type (imbalance, Hawkes, OFI, volatility, dynamics, spread, depth, etc.), by book side (bid, ask, neutral), by price level, or by combinations such as side$\times$type. In the experiments (Section 5.6.2), two groupings are used: by feature type and by side.

**Baseline substitution.** The default baseline mode is the sequence mean: each occluded feature is replaced by its mean value across the $T$ time steps of the input sequence. This preserves the marginal distribution of the non-occluded features better than a zero baseline, which for standardised data would substitute the training-set mean rather than the sample-specific mean.

**Implementation note.** All $K$ occluded variants and the original input are stacked into a single batch of size $K + 1$, passed through the Transformer encoder in one forward call, and scored by the OC-SVM in one call to the decision function. This avoids the $K$-fold overhead of sequential evaluation and makes the computation tractable even for fine-grained groupings with $K > 20$.

**Critical remark.** Because each group is occluded independently (one group at a time), the attributions do not in general sum to $f(\mathbf{x}) - f(\mathbf{x}')$: interaction effects between groups are not captured. The completeness property (3.52) that IG satisfies does not hold for single-group occlusion. A Shapley-value decomposition, which would average over all $2^K$ group subsets, would restore completeness but is computationally prohibitive for the grouping sizes used in practice. The single-group attributions should therefore be interpreted as first-order importance indicators rather than an exact decomposition of the score.

**Comparison to Integrated Gradients.** IG satisfies formal axioms (completeness, sensitivity, implementation invariance) and produces per-feature, per-time-step attributions, but it requires a differentiable end-to-end model and is sensitive to correlated features: when two features are redundant, IG may split credit between them in unintuitive ways [12]. Grouped Occlusion is model-agnostic (applicable to any scoring function, including the OC-SVM dissimilarity score), aggregates over correlated features by design, and produces group-level attributions that are directly interpretable in domain terms. Its limitation is the absence of completeness and its blindness to within-group structure. The two methods are therefore complementary: IG is used where gradient access is available (PNN, PRAE) and Grouped Occlusion where it is not (Transformer+OC-SVM).

## 3.6 Post-Hoc Analysis

The three detection models each produce a scalar anomaly score per event, but they encode different notions of abnormality: dissimilarity from normal LOB dynamics (Transformer+OC-SVM), estimated spoofing profitability (PNN), and departure from the learned data manifold (PRAE). Post-hoc analysis exploits this complementarity in two ways: clustering the anomaly scores to discover recurring anomaly typologies, and examining the microstructural dynamics around individual anomalies to characterise their market impact.

### 3.6.1 Anomaly Clustering

**Input representation.** Each event in the test set is represented by three anomaly scores: the OC-SVM decision function value from the Transformer+OC-SVM pipeline, the spoofing gain from PNN (Section 3.4), and the reconstruction error from PRAE. An event qualifies as anomalous if at least one model flags it, yielding a set of $n_{\text{anom}}$ events. The three raw scores are independently normalised to $[0, 1]$ via min-max scaling over the full test set before restriction to the anomalous subset, so that each axis contributes equally to the distance computation.

**Clustering algorithm.** HDBSCAN [4] is applied to the $n_{\text{anom}} \times 3$ normalised score matrix. Unlike $k$-means, HDBSCAN does not require the number of clusters to be specified *a priori* and can identify clusters of varying density, which is appropriate when different anomaly types have different prevalences. The minimum cluster size is set to

$$m_{\min} = \max\big(5,\ \min(500,\ \lfloor n_{\text{anom}}/20 \rfloor)\big),$$

ensuring that clusters are large enough to permit statistical profiling while adapting to the total anomaly count. The minimum-samples parameter is fixed at 15. Events that do not belong to any cluster are labelled as noise by HDBSCAN's built-in outlier mechanism.

**Cluster evaluation.** Two internal validity metrics are computed on the non-noise subset: the Silhouette score, which measures how similar each point is to its own cluster relative to the nearest neighbouring cluster, and the Davies-Bouldin index, which averages the ratio of within-cluster scatter to between-cluster separation. Higher Silhouette and lower Davies-Bouldin values indicate better-separated clusters.

**Cluster characterisation.** Each cluster is profiled along four axes. First, the *per-model detection rate* records the fraction of the cluster's events that each model independently flags as anomalous. A cluster with a PNN detection rate $\geq 0.5$ is labelled "spoofing-type"; clusters dominated by OC-SVM or PRAE detections are labelled "general". Second, the *dominant trading period* is determined by the most frequent intraday period (first hour, rest of morning, afternoon, or American open) among the cluster's events. Third, the *most discriminating feature* is identified as the feature whose cluster mean deviates most from the global anomaly mean, measured in units of the global standard deviation. Finally, a two-component PCA projection of the three-dimensional score space provides a visual summary of cluster geometry and overlap.

### 3.6.2 Jump Analysis

Where clustering provides a global taxonomy of anomalies, jump analysis examines individual events at the microstructural level to characterise the price and order-book dynamics that coincide with detected anomalies.

**Anomaly selection.** A subset of anomalies is selected for detailed inspection using a three-tier priority scheme based on model agreement. Tier 1 selects events flagged by all three models (unanimous agreement), ranked by the sum of their min-max-normalised scores. If fewer than $N$ events are available, Tier 2 adds events with two-model agreement, and Tier 3 completes the budget with the highest-scoring event per model. The default budget is $N = 6$.

**Context extraction.** For each selected anomaly at row index $i$, a context window of $\pm 100$ events is extracted from the raw LOB data for price and volume diagnostics. A narrower window of $\pm 60$ events is used for feature trajectory analysis, where higher temporal resolution is needed. The sequence length of 25 events used by the detection models defines the mapping from sequence index back to the corresponding raw row in the daily file.

**Statistical jump tests.** Four complementary tests are applied to the context window around each anomaly.

1. **Barndorff-Nielsen bipower variation** [2] decomposes the realised variance of log-returns into a continuous (diffusion) component and a jump component. The bipower variation $\mathrm{BV} = \frac{\pi}{2} \sum_{i=2}^{n} |r_{i-1}|\,|r_i|$ is robust to jumps, so the difference $\mathrm{RV} - \mathrm{BV}$ isolates the jump contribution. A rolling version with window $w = 20$ tracks the jump ratio $(\mathrm{RV} - \mathrm{BV})/\mathrm{RV}$ over time.

2. **Lo-MacKinlay variance ratio test** [8] compares the variance of $q$-period returns to $q$ times the variance of single-period returns. Under a random walk, the variance ratio $\mathrm{VR}(q) = 1$; significant deviations indicate jumps ($\mathrm{VR} > 1$) or mean-reversion ($\mathrm{VR} < 1$). The test is computed separately on the before and after segments with $q = 5$.

3. **CUSUM** (cumulative sum) tracks the running total of deviations of log-returns from their mean. A sharp increase in the CUSUM statistic localises the onset of a regime change, and the index of maximum deviation identifies the most likely change-point.

4. **Welch $t$-test** is applied feature-by-feature to compare the distributions in the before window $[i - 60, i)$ and the after window $[i, i + 60]$. The top $k = 8$ features with the largest absolute $z$-scores at the anomaly time are tested; significant $p$-values indicate a distributional shift that coincides with the detected anomaly.

**Cross-feature lead-lag analysis.** For each selected anomaly, the pairwise correlation matrix of feature changes is computed at lag zero, and a leading-indicator score is derived by averaging the absolute correlations of each feature with all others. Features whose changes precede correlated movements in other features may point to the originating signal behind the anomaly.

**Limitations.** The analysis is descriptive rather than causal: co-occurrence of a price jump and an anomaly flag does not establish that the anomaly caused the jump, nor that the jump is the sole explanation for the model's alert. The sample size ($N = 6$ per training year) is deliberately small to permit detailed visual inspection, but it precludes statistical generalisation. The bipower variation and variance ratio tests assume a semimartingale model for price dynamics (i.e. that the log-price process can be decomposed into a predictable drift, a continuous local martingale, and a pure-jump component), which may not hold at the highest frequencies where microstructure noise dominates.

This chapter has defined the complete methodological pipeline: feature engineering (Section 3.1), three detection models with complementary inductive biases (Section 3.2), anomaly thresholding via extreme value theory and false discovery rate control (Section 3.3), spoofing gain estimation to link statistical anomalies to economic incentives (Section 3.4), two attribution methods for model interpretability (Section 3.5), and post-hoc analysis to cluster anomalies into typologies and examine their microstructural signatures (Section 3.6). Chapter 4 describes the dataset and experimental protocol used to instantiate this pipeline.

# Chapter 4

# Data and Experimental Setup

This chapter describes the dataset, the preprocessing pipeline that transforms raw limit order book (LOB) snapshots into model-ready features, the training and evaluation protocols, and all hyperparameter settings. Every numerical value reported below can be traced to the configuration file `config/default.yaml`, the preprocessing script `scripts/preprocess.py`, the training script `scripts/train.py`, or the test script `scripts/test.py`.

## 4.1 Dataset Description

### 4.1.1 Data Source and Instrument

The dataset consists of full limit order book snapshots for TotalEnergies SE (Reuters ticker `TOTF.PA`), a large-cap equity listed on Euronext Paris. Each snapshot records ten levels of depth on both the bid and ask sides: prices and volumes at levels 1 through 10, yielding $4 \times 10 = 40$ LOB columns per row. A single timestamp column (`xltime`) encodes the observation time as an Excel serial-date fraction, where the fractional part represents the time of day in UTC. The data is event-driven: a new row is generated whenever the LOB state changes, rather than at fixed time intervals.

### 4.1.2 Data Format and Raw Structure

Raw files are delivered as compressed CSV or Parquet files, one per trading day. Each file nominally contains 41 columns (`xltime` plus the 40 LOB fields), although a subset of raw CSV files exhibits a column misalignment artefact: a spurious second column (`Unnamed: 1`) is inserted after `xltime`, shifting all LOB column headers one position to the right and padding the file to 42 columns. This artefact is corrected during preprocessing (Section 4.2).

The naming convention for LOB columns is `{side}-{type}-{level}`, for example `bid-price-3` or `ask-volume-7`.

### 4.1.3 Dataset Statistics

The processed dataset spans four calendar periods of varying length, summarised in Table 4.1.

Table 4.1: Dataset overview by calendar period.

| Period | Date range | Role | Trading days |
|---|---|---|---|
| 2009 | 2009-01-01 to 2009-12-31 | Out-of-sample test | 261 |
| 2010 | 2010-01-01 to 2010-12-31 | Out-of-sample test | 261 |
| 2015 | 2015-01-02 to 2015-02-05 | Training + in-sample test | 25 |
| 2017 | 2017-01-02 to 2017-04-12 | Training + in-sample test | 73 |

Euronext Paris operates a continuous trading session from 09:00 to 17:30 CET. The number of LOB updates per day varies with market activity; typical days contain on the order of tens of thousands of events after market-hours filtering. In total, the processed corpus comprises 620 daily files.

## 4.2 Preprocessing Pipeline

Each raw daily file passes through the following pipeline, implemented in `scripts/preprocess.py`. The steps are applied independently per day; no information leaks across days during preprocessing.

1. **Column alignment.** Files with 42 columns have the spurious column dropped and the remaining 41 columns renamed to `xltime` plus the 40 canonical LOB names. Files with 41 columns but incorrect headers (e.g. data values used as column names in some Parquet exports) are likewise renamed. Files already bearing correct headers are left unchanged.

2. **Numeric coercion and sorting.** All LOB columns and `xltime` are cast to numeric types (non-numeric entries become `NaN`). Rows are sorted chronologically by `xltime`.

3. **Market hours filtering.** Rows outside the interval 09:00–24:00 CET are discarded. The upper bound is deliberately set beyond the 17:30 session close to retain after-hours updates for out-of-sample testing; during training, only the first hour (09:00–10:00) is used (Section 4.3).

4. **Feature engineering.** The seven feature groups described in Section 3.1 are computed on each day's LOB data. Rolling and exponentially weighted computations use a window of $w = 50$ events (configurable via `features.window`). The complete feature vector comprises 89 engineered features per event, organised as follows:

   - Imbalance (2): L1 and L5 order book imbalance.
   - Dynamics (13): mid-price, spread, micro-price deviation, depth ratios, log return, volume deltas, velocity, acceleration, volatility, and net order flows.
   - Elasticity (4): bid and ask sweep-to-fill costs and slopes.
   - Volatility (5): rolling standard deviation of returns at horizons 50 and 100, price range, absolute velocity, inter-event time $\Delta t$ and $\log \Delta t$.
   - Weighted imbalance (3): Tao-style multilevel imbalance with decreasing, increasing, and constant weight schemes across the first five levels.
   - Event flow (18): Poutré-style rapidity features, including cancel and trade sizes, their moving averages (window = 10), cancel and trade rapidities, and price speeds.
   - Hawkes memory and OFI (44): Fabre-Challet limit and market order flows with EWMA at short (halflife = 5) and long (halflife = 50) horizons, deep-order insertions

at level 5, weighted flows with spatial decay ($\eta \in \{0.001, 0.1, 1, 10\}$) and temporal decay ($\beta \in \{10, 100, 1000\}$), and five-level order flow imbalance.

5. **Warmup trim.** The first $n_{\text{warmup}} = 3{,}000$ rows of each day are discarded to eliminate transient effects from rolling and EWMA initialisations.
6. **Quantile clipping.** Each feature column is clipped to its $[0.1\%, 99.9\%]$ quantile range, computed on the post-warmup data of the current day.
7. **Zero-variance removal.** Columns with standard deviation below $10^{-9}$ are dropped.
8. **Output.** The result is saved as a single Parquet file per day, containing `xltime`, the 40 raw LOB columns, and the surviving engineered features (89 in practice).

**Feature scaling.** Before model input, features are further transformed by an empirical Box-Cox scaler (`EmpiricalBoxCoxScaler`), which applies three operations per feature: (i) a positive shift ($x \leftarrow x + |\min(x)| + \epsilon$, with $\epsilon = 10^{-6}$) to ensure strict positivity The scaler is fitted on the training split of the first day only and then applied (transform-only) to all subsequent days, preventing any forward-looking bias.

## 4.3 Training Protocol

Training follows a sequential, day-by-day regime designed to respect the temporal ordering of financial data. Two fully independent training runs are performed: one on the 2015 period and one on the 2017 period.

**Day selection and train/test split.** For a period containing $N$ daily files, the first $N - 12$ files form the training set. The remaining 12 files are held out entirely and subdivided into two evaluation groups:

- The *proximate test set* $\mathcal{T}_A$ (files $N-11$ through $N-4$, nine files): temporally adjacent to the training data, these files may share microstructural characteristics with it (same market regime, similar volatility levels, similar instrument behaviour).
- The *distal test set* $\mathcal{T}_B$ (files $N-3$ through $N$, three files): further from the training boundary and less likely to share these characteristics.

No file appears in more than one partition. Table 4.2 summarises the resulting counts for each training period.

Table 4.2: Train/test split by calendar period.

| Period | $N$ | Training | $\lvert\mathcal{T}_A\rvert$ | $\lvert\mathcal{T}_B\rvert$ | Total test |
|---|---|---|---|---|---|
| 2015 | 25 | 13 | 9 | 3 | 12 |
| 2017 | 73 | 61 | 9 | 3 | 12 |

**Temporal split rationale.** A temporal (chronological) split is used rather than a random split because financial time series exhibit autocorrelation and non-stationarity. A random split would allow information from the test period to influence the training distribution via overlapping market regimes, violating the assumption that the model is evaluated on genuinely unseen data. A temporal split ensures every test observation follows the training period in calendar time.

**Proximity analysis.** Comparing model performance across $\mathcal{T}_A$ and $\mathcal{T}_B$ tests whether the model generalises beyond the training regime or merely interpolates within it. If performance is sta-

ble across both groups, the temporal proximity concern is empirically unfounded. If performance degrades on $\mathcal{T}_B$, this indicates regime sensitivity and must be noted as a limitation. The original protocol treated the nine proximate files as a passive buffer and discarded them; the updated protocol retains them as $\mathcal{T}_A$ and reports their results separately, interpreted with appropriate caution rather than silently excluded. Results on $\mathcal{T}_A$ and $\mathcal{T}_B$ are reported separately in Section 5.4 and discussed in Section 6.2.

**Per-day procedure.** On each training day:

1. Load the processed Parquet file (Section 4.2).
2. Extract the first hour of continuous trading data (09:00–10:00 CET).
3. Split the first hour into alternating 5-minute blocks: train, validation, train, validation, and so on (six blocks of each type).
4. Apply the feature scaler. On day 1, the scaler is fitted on the training split; on all subsequent days, the existing scaler is used in transform-only mode.
5. Construct sliding-window sequences of length $T = 25$ events for the Transformer-based models (Transformer+OC-SVM and PRAE). The PNN, being a single-step predictor, uses only the last time step of each sequence.
6. Train the model with early stopping (patience = 20 epochs, maximum 1,000 epochs). Model weights carry over from the previous day, implementing a form of continual learning.

**OC-SVM fitting.** The Nyström-approximated OC-SVM is fitted once, after all training days are complete. The frozen Transformer encoder re-encodes every training day's first-hour data into 128-dimensional latent representations. The RBF bandwidth is set via the median heuristic:

$$\gamma = \frac{1}{\text{median}_{i \neq j} \|\mathbf{z}_i - \mathbf{z}_j\|^2}.$$

The linear separator in the Nyström feature space is then trained by SGD (learning rate 0.01, 500 epochs).

**PRAE regularisation.** The gate-strength hyperparameter $\lambda_{\text{reg}}$ is selected on the first training day via a grid search around a heuristic initialisation (from `calculate_heuristic_lambda`). Each candidate $\lambda$ is evaluated over 15 training epochs, and the value minimising validation reconstruction error is retained for all subsequent days.

### 4.3.1 Model Hyperparameters

Table 4.3 lists all hyperparameters. Values are sourced from `config/default.yaml` and the constants defined at the top of `scripts/train.py`.

Table 4.3: Model and training hyperparameters.

| Parameter | Value | Source |
|---|---|---|
| *Shared* | | |
| Sequence length $T$ | 25 | `preprocessing.seq_length` |
| Batch size | 64 | `training.batch_size` |
| Optimiser | Adam | `train.py` |
| Learning rate | $10^{-4}$ | `training.learning_rate` |
| Max epochs | 1,000 | `training.epochs` |
| Early stopping patience | 20 | `training.patience` |
| *Transformer + OC-SVM* | | |
| $d_{\text{model}}$ | 128 | `model.transformer.model_dim` |
| Attention heads | 8 | `model.transformer.num_heads` |
| Encoder/decoder layers | 6 | `model.transformer.num_layers` |
| Feed-forward dimension | 512 | `model.transformer.dim_feedforward` |
| Bottleneck $d_{\text{rep}}$ | 128 | `model.transformer.representation_dim` |
| OC-SVM $\nu$ | 0.01 | `model.ocsvm.nu` |
| Nyström components $m$ | 300 | `train.py: NYSTROEM_COMPONENTS` |
| OC-SVM SGD learning rate | 0.01 | `train.py: OCSVM_SGD_LR` |
| OC-SVM SGD epochs | 500 | `train.py: OCSVM_SGD_EPOCHS` |
| OC-SVM $\gamma$ | Median heuristic | `train.py` |
| *PNN* | | |
| Hidden dimension | 64 | `model.pnn.hidden_dim` |
| Output | $(\mu, \sigma, \alpha)$ | Fixed (3 parameters) |
| *PRAE* | | |
| Backbone | Same Transformer arch. | Shared with above |
| Gate noise $\sigma$ | 0.5 | `model.prae.sigma` |
| $\lambda_{\text{reg}}$ | Grid search (day 1) | `model.prae.lambda_reg = null` |

## 4.4 Test Protocol

**Test set composition.** For each trained model (2015-trained or 2017-trained), test scoring is performed on three groups of files:

- **In-sample held-out** ($\mathcal{T}_A \cup \mathcal{T}_B$): the 12 held-out files from the training period, further subdivided into the proximate set $\mathcal{T}_A$ (9 files) and the distal set $\mathcal{T}_B$ (3 files) as described in Section 4.3.
- **Cross-year**: all days from the other within-decade period. For a 2015-trained model, this is the full 2017 corpus (73 days), and conversely.
- **Out-of-sample**: all days from 2009 (261 days) and 2010 (261 days), which predate both training periods.

This design enables in-distribution evaluation (with proximity stratification), cross-period transfer evaluation, and out-of-distribution evaluation.

**Scoring.** Unlike training, which uses only the first hour, test scoring operates on the full continuous trading session (09:00–17:30 CET). Four intraday periods are defined for stratified analysis: first hour (09:00–10:00), rest of morning (10:00–12:00), afternoon (12:00–15:30), and American open (15:30–17:30).

**Anomaly classification.** Each pipeline applies its dedicated threshold mechanism:

- **Transformer + OC-SVM**: an event is flagged as anomalous if the dissimilarity score $d(\mathbf{z}) = -f(\mathbf{z})$ (Eq. 3.28) exceeds the baseline threshold $\tau_0$ (Eq. 3.29), computed as the $(1-\epsilon)$-quantile of training dissimilarity scores with $\epsilon = 0.01$. When a calibrated EVT threshold is available, it replaces $\tau_0$.
- **PNN**: the spoofing gain $\Delta\mathcal{C}$ is computed from the predicted skew-normal parameters $(\mu, \sigma, \alpha)$ and the observed spread, using $Q = 4{,}500$ (large order), $q = 100$ (small order), bid displacement $\delta_a = 0$, ask displacement $\delta_b = 0.01$, maker fee 0, and taker fee 0.0008 (Euronext Paris large-cap schedule). An event is flagged if $\Delta\mathcal{C} > 0$.
- **PRAE**: the Rolling False Discovery Rate method (Section 3.3.3) is applied to the reconstruction error, with window size $w = 500$ and significance level $\alpha = 0.05$.

**Evaluation metric.** Since no ground-truth manipulation labels are available, model comparison relies on internal consistency metrics (anomaly rates, cross-model agreement, score distributions) rather than supervised accuracy. Where a reference labelling is available for benchmarking, the $F_\beta$ score with $\beta = 4$ is used, weighting recall four times higher than precision, as missing a true manipulation event is considered more costly than a false alarm.

## 4.5 Compute Environment

Model training is executed on the Ruche HPC cluster (Université Paris-Saclay / CentraleSupélec mesocentre), using a single NVIDIA A100-40 GB GPU node with 8 CPU cores and 40 GB of host RAM. The software environment consists of Python 3.10, PyTorch with CUDA 12.1, and standard scientific libraries (NumPy, pandas, scikit-learn, SciPy). Post-training analysis (notebooks, visualisation, sensitivity analysis) is performed locally on CPU.

# Chapter 5

# Results

## 5.1 Model Diagnostics

Figure 5.1: Training and validation loss curves for each detection model.

Figure 5.2: Anomaly score distributions on the 2015 and 2017 test data for each model.

Figure 5.3: Dynamic threshold evolution for SPOT, DSPOT, and RFDR on a representative test day.

Figure 5.4: HDBSCAN clusters in the anomaly-score space (PCA projection).

Figure 5.5: Integrated Gradient attribution: top-20 most influential features per model.

### 5.1.1 Transformer + OC-SVM

### 5.1.2 PNN

### 5.1.3 PRAE

## 5.2 Anomaly Detection on Test Data

### 5.2.1 Anomaly Rates by Model and Time Period

### 5.2.2 Score Time Series

### 5.2.3 Model Agreement

## 5.3 Threshold Comparison

## 5.4 Proximity Analysis

## 5.5 Spoofing Gain Analysis

## 5.6 Sensitivity and Feature Attribution

### 5.6.1 Integrated Gradients: PNN and PRAE

### 5.6.2 Grouped Occlusion: Transformer + OC-SVM

### 5.6.3 Cross-Model Feature Importance

## 5.7 Anomaly Clustering

## 5.8 Jump Analysis

## 5.9 Cross-Year Robustness

# Chapter 6

# Discussion

## 6.1 Limitations

## 6.2 Generalization and Regime Sensitivity

## 6.3 Comparison of Detection Approaches

## 6.4 Interpretation of Detected Anomalies

# Chapter 7

# Conclusion

## 7.1 Summary of Contributions

## 7.2 Future Work

# Bibliography


[1] August A. Balkema and Laurens de Haan. "Residual Life Time at Great Age". In: *The Annals of Probability* 2.5 (1974), pp. 792–804.

[2] Ole E. Barndorff-Nielsen and Neil Shephard. "Power and Bipower Variation with Stochastic Volatility and Jumps". In: *Journal of Financial Econometrics* 2.1 (Jan. 2004), pp. 1–37. ISSN: 1479-8409. DOI: 10.1093/jjfinec/nbh001. eprint: https://academic.oup.com/jfec/article-pdf/2/1/1/2840679/nbh001.pdf. URL: https://doi.org/10.1093/jjfinec/nbh001.

[3] Yoav Benjamini and Yosef Hochberg. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing". In: *Journal of the Royal Statistical Society: Series B (Methodological)* 57.1 (1995), pp. 289–300.

[4] Ricardo Campello, Davoud Moulavi, and Joerg Sander. "Density-Based Clustering Based on Hierarchical Density Estimates". In: vol. 7819. Apr. 2013, pp. 160–172. ISBN: 978-3-642-37455-5. DOI: 10.1007/978-3-642-37456-2_14.

[5] Timothée Fabre and Damien Challet. *Learning the Spoofability of Limit Order Books With Interpretable Probabilistic Neural Networks*. 2025. arXiv: 2504.15908 [q-fin.TR]. URL: https://arxiv.org/abs/2504.15908.

[6] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[7] Ofir Lindenbaum, Yariv Aizenbud, and Yuval Kluger. *Probabilistic Robust Autoencoders for Outlier Detection*. 2022. arXiv: 2110.00494 [cs.LG]. URL: https://arxiv.org/abs/2110.00494.

[8] Andrew W. Lo and A. Craig MacKinlay. "Stock Market Prices do not Follow Random Walks: Evidence from a Simple Specification Test". In: *The Review of Financial Studies* 1.1 (1988), pp. 41–66. ISSN: 08939454, 14657368. URL: http://www.jstor.org/stable/2962126 (visited on 04/04/2026).

[9] James Pickands. "Statistical Inference Using Extreme Order Statistics". In: *The Annals of Statistics* 3.1 (1975), pp. 119–131.

[10] Cédric Poutré, Didier Chételat, and Manuel Morales. "Deep Unsupervised Anomaly Detection in High-Frequency Markets". In: *The Journal of Finance and Data Science* 10 (Apr. 2024), p. 100129. DOI: 10.1016/j.jfds.2024.100129.

[11] Alban Siffer et al. "Anomaly Detection in Streams with Extreme Value Theory". In: *KDD 2017 - Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Halifax, Canada, Aug. 2017. DOI: 10.1145/3097983.3098144. URL: https://hal.science/hal-01640325.

[12] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. *Axiomatic Attribution for Deep Networks*. 2017. arXiv: 1703.01365 [cs.LG]. URL: https://arxiv.org/abs/1703.01365.

[13] Xuan Tao et al. *On Detecting Spoofing Strategies in High Frequency Trading*. 2020. arXiv: 2009.14818 [q-fin.TR]. URL: https://arxiv.org/abs/2009.14818.

[14] Ashish Vaswani et al. *Attention Is All You Need*. 2017. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

# Appendix A

# Feature Catalogue

# Appendix B

# Hyperparameter Tables

# Appendix C

# Additional Figures

## C.1 Score Distributions per Test Day

## C.2 LOB Snapshots for Selected Anomalies

## C.3 Feature Evolution Around Anomalies

# Appendix D

# Threshold Method Details

# Appendix E

# Spoofing Gain Derivation